Reading Guide: Package Hallucination & Slopsquatting

Package hallucination and slopsquatting in LLM coding assistants: measurement, exploitability and lightweight defences

Generated 2026-11-12

Overview

What this research is about

AI coding assistants such as ChatGPT, Copilot, Claude Code and Cursor write code for you, and that code often relies on packages. A package (also called a dependency) is a ready-made piece of code that other people wrote and published. You download it with a command such as pip install (for Python) or npm install (for JavaScript). These commands fetch the package from a package registry, a public online store of packages: PyPI for Python, npm for JavaScript, crates.io for Rust.

Sometimes the AI recommends a package that does not exist. This is called package hallucination. On its own, it only causes an error. The danger starts when an attacker notices the invented name, registers it on the registry, and puts malware inside. The next developer who trusts the AI and runs the install command gets the malware. This attack is called slopsquatting. It is a new kind of software supply-chain attack, which means an attack that reaches you through third-party code your project depends on.

This project asks how often package hallucination happens, how easy it is to exploit, and which defences work. The aim is to design a small study that you can run on an ordinary laptop. We read and analysed 17 papers in full.

The five research questions (RQs), in plain words:

1. **RQ1. How often?** How often do AI models and coding agents invent package names, and what makes it more or less likely (model, programming language, prompt, model compression, agent use)?
2. **RQ2. Exploitability.** Are invented names predictable enough for an attacker to use, and has anyone actually registered them?
3. **RQ3. Defences.** Which defences exist, how well do they work, what do they cost, and can they run on a normal laptop?
4. **RQ4. Measurement.** How do studies decide that a name is invented, and which of their choices can distort the numbers?
5. **RQ5. Gaps.** What has not been tested yet that a small laptop study could address?

How to use this guide

The guide has three parts:

- **This overview.** Read it first. It gives the main answers, a map of all 17 papers, the five papers to read first, and where studies disagree.
- **The paper briefs.** One short brief per paper, in the same order as the paper map below. Read the five "read first" briefs, then the briefs for the group closest to your interest.
- **Research gaps, glossary and notes.** The last part explains the open questions (gaps G1 to G5), the study we recommend, the safety rules for that study, a next-steps checklist, a glossary of every technical term, and how this guide was made, including what is missing from it.

If a word is new to you, look it up in the glossary at the end.

Evidence labels. Each important statement carries one label:

- **[V] Verified.** We checked it in the paper's own data, tables or results.
- **[A] Author claim.** The authors say it, but we could not check it ourselves (for example, a claim that something is not covered by their study).
- **[I] Interpretation.** Our own reasoning, built from the cited papers. Treat it as an argument, not a fact.

Citations. A citation looks like this: [churilov2026range, p. 12]. The first part (churilov2026range) is the paper's key, which is also the name of its folder in the project. "p. 12" is the page of the PDF we hold. That example points to the page where the Range Shrinks paper reports how many invented names were later registered. Page numbers refer to our PDF copies. For six of the seven peer-reviewed papers our copy is the arXiv version, so page numbers may differ from the publisher's version.

Peer-reviewed vs preprint. Peer review means independent experts checked a paper before a journal or conference published it. A preprint is a paper shared publicly (often on arXiv) before or without that check.

- 7 of our 17 papers are peer-reviewed: spracklen2024we, zhao2025hfuzzer, twist2026library, zhang2025llm, djire2026evaluating, liu2026packmonitor, and the background book chapter ohm2020backstabbers.
- 10 are preprints: churilov2026range, okwor2026hallucination, krishna2025importing, haque2025secure, zheng2026when, dasu2026refusing, liu2026mitigating, spracklen2026llm, raj2026names and mirandapena2026empirical.
- Special case one: our database lists spracklen2024we as a preprint, but it appeared at USENIX Security 2025, a top peer-reviewed security conference [churilov2026range, p. 1]. We treat it as peer-reviewed. We read its arXiv version.
- Special case two: okwor2026hallucination is a self-published report on Zenodo (an open research archive). It is openly licensed but has had no peer review.
- Special case three: mirandapena2026empirical may have been accepted by a journal, but we could not confirm this, so it stays a preprint.
- We use preprints but give them less weight. The paper map marks each paper's status.

Evidence strength. We rate how much we trust each answer:

- **Strong:** enough data, sound design, and several studies agree.
- **Moderate:** sound design, but a single study or limited data.
- **Weak:** small or unclear samples, design problems, or preprints nobody has repeated.

In the project's evidence table, every finding from a preprint was lowered by one step.

Main findings at a glance

> **In one sentence:** every AI model tested invents some package names, many of those names come back again and again, attackers have so far claimed few of them, and the

> defences that work best need data-centre hardware, while the cheap defences that would work on a laptop have barely been tested.

RQ1. How often do models invent package names?

> **Answer:** in every model tested, but how often depends heavily on the model, the prompt and how each study counts. Evidence strength: moderate that it is real and depends on the model; weak for most other drivers.

- **[V]** Every study that measured it found package hallucination in every model it tested. This covers 16 models from 2023–24 [spracklen2024we, p. 7], five frontier models from 2026 [churilov2026range, p. 6], and Rust and Go as well as Python and JavaScript [zheng2026when, p. 5; haque2025secure, p. 6].
- **[V]** The size of the problem depends on the model class and on what is counted. Across 16 models from 2023–24, 19.7% of 2.23 million package mentions were invented [spracklen2024we, p. 7]. For five 2026 frontier models it was 3.63–5.02% of mentions (preprint) [churilov2026range, p. 6]. For eight small open models with 1–8 billion parameters it was 16.1–47.9% [djire2026evaluating, p. 5].
- **[V]** In Rust, about 22% of crate references (a crate is a Rust package) were invented, and nearly half of these were confusions with Rust's built-in modules (preprint) [zheng2026when, p. 5].
- **[V]** The prompt matters a lot. With ordinary requests, library-name hallucination was near zero (at most 0.31% of responses) across seven commercial models [twist2026library, p. 5]. Asking for libraries "from 2025" raised it to as much as 84.74% of tasks for one model [twist2026library, p. 5].
- **[V]** When the user types a misspelled or fake library name, models often go along with it: up to 79.44% of tasks for multi-character typos and 99.22% for fake names [twist2026library, p. 6]. Two other papers confirm this tendency [djire2026evaluating, p. 9; dasu2026refusing, p. 8].
- **[V]** Other suspected drivers are contested or rest on one study. The language that comes out worse flips between studies (see C1 below). The one significant size effect relies on repeated data points that are not independent [krishna2025importing, p. 6]. Temperature mattered in one study [spracklen2024we, p. 8] but not in another [zheng2026when, p. 6]. Quantization (compressing a model so it runs on smaller hardware) was tested in only one study, on Go, and its tables are not consistent [haque2025secure, p. 6].
- **[A]** No analysed paper measures AI coding agents that run install commands by themselves [churilov2026range, p. 14].

RQ2. What makes invented names exploitable?

> **Answer:** many invented names are predictable enough to squat, but attackers have so far registered few of them. Evidence strength: moderate for predictability; weak for real-world

> exploitation.

- **[V]** Names come back. 43% of invented names reappeared in all 10 repeated queries [spracklen2024we, p. 9]. 104 names were invented by all five 2026 frontier models, and 54 of these are one character away from a real package (preprint) [churilov2026range, p. 9].
- **[V]** Most names are still specific to one model: 81% of distinct invented names came from a single model in 2023–24 [spracklen2024we, p. 10], and 450 of 455 newly found names in a 2026 study came from a single model [okwor2026hallucination, p. 4].
- **[V]** Many invented names do not look like a real package. Only 13.4% are within one or two character changes of a real name, and 48.6% are six or more changes away [spracklen2024we, p. 11]. A simple "looks like a famous package" check would miss these.
- **[V]** Real-world claiming has been rare so far. 25 of 2,282 invented names (about 1.1%) were registered within about six months, one of them with malware, and at least 23 suggested names had earlier hosted malware (preprint) [churilov2026range, p. 12].
- **[V]** Of 149 publicly listed invented names, 22 were registered, none with malicious content, and 19 were placeholders apparently registered by defenders (self-published preprint) [okwor2026hallucination, p. 5].
- **[V]** The route for the malware is well established. 56% of 174 real malicious packages ran their code during installation, and 96% of the PyPI ones did [ohm2020backstabbers, p. 11]. Install-time code execution through pip and npm was reproduced offline [okwor2026hallucination, p. 7].

RQ3. Which defences work, and can they run on a laptop?

> **Answer:** the strongest defences need access to the model's internals and data-centre GPUs; the cheap defences that work with any AI tool are only partly effective and barely tested. Evidence strength: weak to moderate.

- **[V]** White-box defences (which need the model's internal weights or scores) give the largest reductions. PackMonitor, which blocks any token that cannot form a name on the PyPI list, reached 0% hallucination on five 7–8B models with 7–28% extra time [liu2026packmonitor, p. 13; liu2026packmonitor, p. 14].
- **[V]** Model editing cut the package hallucination rate from 0.4421 to 0.1529 on unseen high-risk prompts (preprint) [liu2026mitigating, p. 7]. Unlearning cut it from 21.23% to 2.56% for a 7B model (preprint) [spracklen2026llm, p. 10].
- **[V]** These defences ran on data-centre hardware: 8 A100 GPUs, or 12–24-hour runs on AMD MI210/MI300 accelerators [liu2026packmonitor, p. 12; spracklen2026llm, p. 19]. Retraining can also damage coding skill: in one test the score fell from 51.4% to 25.3% [spracklen2024we, p. 13].
- **[V]** Black-box defences (which only need the model's text output) are cheap but inconsistent. Retrieval-augmented generation (RAG) lowered the average rate from 29.7% to 18.1% in one study but raised it by 12.7 percentage points for JavaScript [djire2026evaluating, p. 6]. Greedy decoding was the best simple trade-off for small models [djire2026evaluating, p. 8].

- **[V]** Self-check prompts and an optional PyPI lookup tool reduced hallucination without removing it, and one model called the tool on only 7.89% of responses [twist2026library, p. 18; twist2026library, p. 21].
- **[A]** Checking only whether a name exists on the registry is not enough, because an attacker can register the name first [spracklen2024we, p. 12]. **[I]** PackMonitor's filter lets through any registered name, including squatted ones [liu2026packmonitor, p. 6].
- **[V]** The only tool-side guard in our set finished cleanly on 228 of 300 hand-written prompts, in a small evaluation (preprint) [raj2026names, p. 7]. **[A]** Its most important signal, the number of maintainers, is not available through the public PyPI interface [raj2026names, p. 13].
- **[V]** Only static code analysis was run on a laptop-class machine (Apple M2, 24 GB) [mirandapena2026empirical, p. 17]. One of its tools, Pyright, caught between 14% and 85% of library hallucinations, depending on model and benchmark [mirandapena2026empirical, p. 8].

RQ4. How do studies measure package hallucination, and what distorts it?

> **Answer:** every study counts differently, so their numbers cannot be put side by side. Evidence strength: moderate.

- **[V]** Studies differ in where prompts come from, how names are pulled out of answers, whether they check names against a saved registry list or the live registry, how they treat built-in modules, and whether they count per name, per answer or per prompt. Because of this, rates cannot be pooled or ranked across papers [spracklen2024we, p. 6; krishna2025importing, p. 4; churilov2026range, p. 5].
- **[V]** Counting Python's standard library (built-in modules that never need installing) as invented adds 7.6 to 9.4 percentage points [djire2026evaluating, p. 5].
- **[V]** Import names of real packages (for example cv2, which you install as opencv-python) were the largest class of false alarms. A live re-check of the registry lowered rates by about a fifth [churilov2026range, p. 10].
- **[V]** In a small study, most early "hallucinations" were really errors in the tool that pulled names out of the answers [raj2026names, p. 7].
- **[V]**/**[A]** Further threats: statistics built on data points that are not independent [krishna2025importing, p. 6]; model size mixed up with quantization [dasu2026refusing, p. 10]; and going along with a planted fake name counted as invention [twist2026library, p. 6].
- **[V]** Some studies judge "does not exist" against the task's own installed environment, not a registry, so their numbers do not measure slopsquatting exposure [zhang2025llm, p. 6; mirandapena2026empirical, p. 4].

RQ5. What is still missing?

> **Answer:** five gaps passed a recorded novelty check on 2026-11-12. The recommended project combines three of them: first a shared way to score names (G3), then a measurement of AI coding agents (G1), then a test of simple pre-install guards (G2).

> Evidence strength: preliminary to moderate, depending on the gap.

- **[A]** Five analysed papers name AI coding agents as untested [churilov2026range, p. 14; dasu2026refusing, p. 10; twist2026library, p. 9; liu2026packmonitor, p. 19; okwor2026hallucination, p. 8].
- **[I]** The five gaps are: how often coding agents actually try to install invented names (G1); an independent comparison of laptop-friendly pre-install guards (G2); a validated, shared measurement rule (G3); the effect of compressed local models on PyPI and npm (G4); and long-term watching of published names (G5).
- **[I]** Each gap is phrased as "we found no paper that ...", not "this has never been studied". The gaps section at the end explains each one, with a study design for a beginner.

Paper map

The papers are grouped by approach, following the methodology comparison:

- **M. Measurement studies.** Ask models many coding questions, pull the package names out of the answers, and check them against the registry.
- **T. Testing and benchmarks.** Search on purpose for prompts that make models invent names, or build fixed test sets.
- **D. Defences inside or around the model.** Change the model's weights, change how it picks the next word, or give it extra information.
- **S. Tool-side and static checks.** Check names or code outside the model, without changing the model.
- **B. Background.** Real supply-chain attacks from before AI coding assistants.

Relevance (high, medium, low) is our rating of how close each paper is to the project question.

#	Group	Paper (short title)	Year	Type	Status	What it is about	Relevance
1	M	We Have a Package for You! (spracklen2024we)	2024	Large measurement + mitigation test	Peer-reviewed (USENIX Security 2025; our copy is arXiv)	16 models, 2.23 million package mentions in Python and JavaScript; how often names repeat	high
2	M	The Range Shrinks, the Threat Remains (churilov2026range)	2026	Preregistered replication + natural experiment	Preprint	Repeats the study above on five 2026 frontier models with a live registry check; tracks which names get registered	high
3	M	From Hallucination	2026	Registration	Preprint (self-publis	Do invented names get claimed on PyPI	medium

		to Registration (okwor2026hallucination)		measurement	hed on Zenodo)	and npm? Who claims them?	
4	M	Importing Phantoms (krishna2025importing)	2025	Measurement + correlations	Preprint	11 models in JavaScript, Python and Rust; links to model size and coding scores	high
5	M	Secure or Suspect? (haque2025secure)	2025	Measurement	Preprint	Effect of model compression (quantization) on invented Go packages	high
6	M	When LLMs Invent Rust Crates (zheng2026when)	2026	Measurement + prompt mitigation	Preprint	Invented Rust packages; effects of model size and temperature	high
7	M	LLM Hallucinations in Practical Code Generation (zhang2025llm)	2025	Taxonomy study	Peer-reviewed (Proc. ACM Softw. Eng., ISSTA; our copy is arXiv)	Types of code errors in real projects; missing packages are a very small share	low
8	T	HFuzzer (zhao2025hfuzzer)	2025	Testing tool + experiment	Peer-reviewed (ASE 2025; our copy is arXiv)	One AI writes tasks that make another AI invent Python packages	high
9	T	Refusing the Impossible (dasu2026refusing)	2026	Benchmark	Preprint	Do 17 open models refuse requests that cannot be done, such as using a package that does not exist?	high
10	T	Library Hallucinations in LLM-Generated Code (twist2026library)	2026	Controlled experiments + benchmark	Peer-reviewed (EMNLP 2026)	Which prompt wordings, typos and fake names trigger invented libraries	high
11	D	Evaluating Inference-Time	2026	Comparative	Peer-reviewed (ASE	Eight cheap defences on eight	high

		Defenses (djire2026evaluating)		experiment	2026; our copy is arXiv)	small open models in four languages; counting biases	
12	D	PackMonitor (liu2026packmonitor)	2026	Technique + experiment	Peer-reviewed (Proc. ACM Softw. Eng.; our copy is arXiv, under a different title)	Blocks invented names while the model writes a pip install command	high
13	D	Model Editing against Package Hallucination (liu2026mitigating)	2026	Technique + experiment	Preprint	A small add-on to the model's weights that targets invented names	medium
14	D	LLM Ghostbusters (spracklen2026llm)	2026	Technique + experiment	Preprint	"Unlearning" invented names by retraining the model	medium
15	S	Names Can Hurt (raj2026names)	2026	System + small evaluation	Preprint	A registry check plus a metadata classifier that flags risky names	high
16	S	Static Analysis for Library Hallucinations (mirandapena2026empirical)	2026	Comparison of detection tools	Preprint (journal acceptance not confirmed)	Code-checking tools that catch invented library features; run on a laptop	medium
17	B	Backstabber's Knife Collection (ohm2020backstabbers)	2020	Dataset of real attacks + taxonomy	Peer-reviewed (book chapter, Lecture Notes in Computer Science; our copy is arXiv)	174 real malicious npm, PyPI and RubyGems packages and how they attacked	medium

Reading order: the five papers to read first

Start with these five. Together they cover how to measure the problem, why it matters, and what a laptop defence would need to handle.

1. **The Range Shrinks, the Threat Remains (churilov2026range, preprint).** The most careful measurement in the set. It preregistered its analysis (published the plan before collecting data), re-checked names against the live registry, reported confidence intervals, and tracked which invented names were later registered [churilov2026range, p. 5; churilov2026range, p. 12]. Read it as the model for a careful protocol.
2. **We Have a Package for You! (spracklen2024we, peer-reviewed).** The founding large-scale study. Later papers reuse its prompts, and it gives the repetition and lookalike results behind the attack model [spracklen2024we, p. 9; spracklen2024we, p. 11].
3. **Library Hallucinations in LLM-Generated Code (twist2026library, peer-reviewed).** A controlled study showing which prompts trigger invented libraries, and that a model rarely uses an optional lookup tool [twist2026library, p. 5; twist2026library, p. 8]. **[I]** This is the main argument for a check that runs every time, not one the model may skip.
4. **Evaluating Inference-Time Defenses (djire2026evaluating, peer-reviewed).** Compares cheap defences on small open models of the kind you can run on a laptop, and measures how much counting built-in modules distorts the numbers [djire2026evaluating, p. 5; djire2026evaluating, p. 6].
5. **Names Can Hurt (raj2026names, preprint).** The closest existing laptop-style guard. It is most useful for its pitfalls: name-extraction errors, import-name confusion, and a key signal that is hard to obtain [raj2026names, p. 7; raj2026names, p. 9; raj2026names, p. 13].

Where studies disagree

All explanations here are our interpretation **[I]**. None has been tested, and we do not pick a side. Full entries are in the project's evidence matrix.

- **C1. Which language is worse, Python or JavaScript?** JavaScript was worse in one study [spracklen2024we, p. 7], Python in two others [krishna2025importing, p. 8; churilov2026range, p. 7]. **[I]** The first study takes names from install commands and from lists the model is asked for; the other two read the import lines in the code. Python import names often differ from install names (cv2 versus opencv-python), so reading imports can inflate Python's rate.
- **C2. Which prompts are worse, Stack Overflow questions or AI-written prompts, on the same prompt set?** Stack Overflow prompts gave higher rates in 2024 [spracklen2024we, p. 20]; AI-written prompts gave higher rates in 2026 [churilov2026range, p. 7]. **[I]** The models, the name extraction and the year all differ. This is the cleanest contradiction in the set, and re-scoring one set of answers with both methods would test it.
- **C3. Do bigger models invent fewer names?** Yes in one study, but its statistics rest on data points that are not independent [krishna2025importing, p. 6]. No size effect for Rust in another [zheng2026when, p. 6], and a newer 8B model produced more unique invented names than its predecessor [zhao2025hfuzzer, p. 7]. **[I]** Size is mixed up with quantization in several studies [spracklen2024we, p. 6; dasu2026refusing, p. 10].
- **C4. Does temperature matter?** Rates rose sharply above temperature 1 in one study [spracklen2024we, p. 8] and did not change for Rust in another [zheng2026when, p. 6]. **[I]** Different languages and counting rules; unresolved.

- **C5. Is a registered import name valid or a squat?** One paper counts it as valid [twist2026library, p. 4]. Another flags pil and faiss as squats [raj2026names, p. 8]. A third probably counts unregistered import names such as cv2 as invented [djire2026evaluating, p. 6]. **[I]** The field has no shared table that maps import names to install names.
- **C6. Does RAG help?** Large gains in Python in one study [spracklen2024we, p. 13], little effect in Rust [zheng2026when, p. 9], and small increases in two other setups [liu2026packmonitor, p. 13; djire2026evaluating, p. 6]. **[I]** "RAG" means a different retriever and document collection in each paper.
- **C7. How often do invented names get registered? (compatible, not a real contradiction).** 25 of 2,282 names over about six months [churilov2026range, p. 12], against 0 of 1,641 unpublished names after four days [okwor2026hallucination, p. 5]. **[I]** The time windows and the sources of names differ, so both can be true.

Paper Summaries

Each of the 17 analysed papers gets a 1–2 page brief, in the same order as the paper map. Every brief opens with a one-line takeaway and a fact table, then explains in plain words what the paper did, how, with what data, what it found, its weaknesses, which of our gaps (G1–G5) it leaves open, why it matters for your project, and how it connects to the other papers.

We Have a Package for You! A Comprehensive Analysis of Package Hallucinations by Code Generating LLMs

> **Takeaway:** This is the founding large-scale study. Across 16 code models from 2023–24, about one in five suggested package names did not exist, and many invented names came back again and again. That repeatability is what makes "slopsquatting" (registering an invented name to plant malware) a realistic attack.

Item	Detail
Authors	Joseph Spracklen, Raveen Wijewickrama, A H M Nazmus Sakib, Anindya Maiti, Bimal Viswanath, Murtuza Jadliwala
Year · Venue	2024 (arXiv) · published at USENIX Security 2025
Status	Peer-reviewed (USENIX Security 2025); our copy is the arXiv v3 version, so page numbers follow that copy
Identifier	DOI 10.48550/arxiv.2406.10279 (arXiv 2406.10279)
Paper type	Empirical measurement study plus mitigation experiment
Relevance to us	High — RQ1, RQ2, RQ3, RQ4

In plain words

When you ask an AI coding assistant for code, it often tells you to install extra packages. A package is a ready-made code library downloaded from a public registry, such as PyPI for Python or npm for JavaScript. Sometimes the model (a large language model, or LLM) names a package that does not exist. This is a package hallucination. If an attacker registers that invented name and uploads malware, anyone who follows the model's advice installs the malware. The authors asked how often this happens, what makes it worse, what the invented names look like, and whether it can be reduced [spracklen2024we, p. 4]. Their attacker can query the same models, check names against the registry and register the missing ones, while victims install suggested packages without checking [spracklen2024we, p. 4].

How they did it

1. Build prompts. One set came from Stack Overflow questions; the other was written by the Llama-2 70B model from the descriptions of the 5,000 most-downloaded PyPI and npm packages [spracklen2024we, p. 5].
2. Generate code. 16 models wrote Python and JavaScript code, giving 30 model-language tests [spracklen2024we, p. 6].
3. Collect package names in three ways: install commands the model wrote on its own, asking the model which packages its code needs, and asking which packages would solve the task. Reading import lines was rejected, because module names do not map one-to-one onto package names [spracklen2024we, p. 6].

4. Decide what is hallucinated. A name counts as hallucinated if it is missing from a master list of PyPI or npm names taken on 10 January 2024 [spracklen2024we, p. 7].
5. Probe behaviour. Extra experiments varied temperature (a setting that controls how random the output is), repeated prompts 10 times, and asked models to spot their own invented names [spracklen2024we, p. 8; spracklen2024we, p. 9; spracklen2024we, p. 10].
6. Test fixes on two open models: RAG (retrieval-augmented generation, which adds looked-up facts to the prompt), self-refinement (the model re-checks its answer), and fine-tuning (extra training on correct examples) [spracklen2024we, p. 12; spracklen2024we, p. 13].

Data and setup

- 576,000 code samples and 2.23 million package mentions [spracklen2024we, p. 6; spracklen2024we, p. 7]. The contributions list says 586,000 samples instead, and the paper does not explain the difference [spracklen2024we, p. 2].
- Models from 2023 to early 2024, including GPT-4 Turbo, GPT-3.5, CodeLlama and DeepSeek Coder. Open models were run in GPTQ-quantized (compressed) form [spracklen2024we, p. 6].
- Temperature 0.7 for code and 0.01 for the package questions [spracklen2024we, p. 19]. Experiments ran on a GPU cluster [spracklen2024we, p. 18].
- Name normalisation (treating variants such as my-pkg and my_pkg as the same) and standard-library exclusion are not reported.
- Prompts and code were released, which a later replication confirms [churilov2026range, p. 15]. The list of hallucinated names is withheld [spracklen2024we, p. 14].

Key findings

- **[V]** 440,445 of 2.23 million package mentions (19.7%) were hallucinated, covering 205,474 unique invented names [spracklen2024we, p. 7].
- **[V]** GPT models averaged 5.2% and open-source models 21.7% [spracklen2024we, p. 7]. **[I]** Our recomputation shows these are averages of per-test rates, not pooled rates, and GPT-4 Turbo's "lowest" 3.59% is its Python-only figure [spracklen2024we, p. 20].
- **[V]** Invented names repeat: in 10 re-runs of 500 prompts, 43% of names came back every time and 39% never came back [spracklen2024we, p. 9].
- **[V]** 81% of distinct invented names came from only one model [spracklen2024we, p. 10].
- **[V]** They are mostly not typos: only 13.4% are within 1–2 character edits of a real package, while 48.6% are 6 or more edits away [spracklen2024we, p. 11].
- **[V]** Rates rise with temperature, sharply above 1; GPT-3.5 reached 31.8% at maximum temperature [spracklen2024we, p. 8].
- **[V]** Fine-tuning cut DeepSeek 6.7B from 16.14% to 2.66%, but its HumanEval coding score fell from 51.4% to 25.3%. RAG cut CodeLlama 7B from 26.28% to 13.40% [spracklen2024we, p. 13].
- **[V]** Deleted PyPI packages explain only 0.17% of names, and 8.7% of invented Python names are real npm packages [spracklen2024we, p. 11].

Limitations and weaknesses

- **[A]** The master list may already contain squatted names, so the rates are a lower bound [spracklen2024we, p. 7].
- **[A]** For ethical reasons no names were registered, so real exploitation was not shown [spracklen2024we, p. 14].
- **[I]** The rate mixes install commands with direct "which packages?" answers [spracklen2024we, p. 6]. So it is not the same as "invented dependency in code" and cannot be compared directly with studies that read import lines.
- **[I]** Headline numbers mix pooled and averaged rates [spracklen2024we, p. 20].
- **[I]** Each condition was run once, with no confidence intervals or significance tests, so small differences cannot be judged [spracklen2024we, p. 13].
- **[I]** The fine-tuning data came from the same prompt pool used for evaluation, so the 2.66% may be partly in-sample [spracklen2024we, p. 12].
- **[I]** Several internal inconsistencies (sample count, heuristic numbering between main text and appendix) lower confidence in exact numbers [spracklen2024we, p. 2; spracklen2024we, p. 18].

Gaps and future work this paper leaves open

- **[A]** The authors call for finding root causes, building code-specific mitigations and fine-tuning that keeps code quality [spracklen2024we, p. 13].
- **[A]** G2: they argue a plain existence check is an ineffective defence, because an attacker could publish a hallucinated name immediately [spracklen2024we, p. 12]. Which extra signals work is still untested.
- **[I]** G1: only chat-style generation was studied. The 43% persistence [spracklen2024we, p. 9] makes predictable installs by coding agents plausible, but nobody has measured them.
- **[I]** G3: its way of collecting names differs from later studies, which is a main reason rates across papers cannot be compared.
- **[I]** G4: all open models were quantized [spracklen2024we, p. 6], but quantization was never varied.
- **[A]** G5: names were not registered and the list was withheld [spracklen2024we, p. 14], so registration over time was not tracked.

Why it matters for your research

This is the baseline every later paper compares against. Its prompt corpus is public and reusable [churilov2026range, p. 4]. Two practical lessons: sample each prompt several times, because about four in ten names repeat every time; and do not rely on an existence check alone. When you quote 19.7%, add that it concerns 2023–24 models, counts per mention, and mixes name sources.

Connections to other papers

- **[V]** churilov2026range re-ran the same prompts on five 2026 frontier models and found 3.63–5.02% of mentions hallucinated [churilov2026range, p. 6]. **[I]** It parses imports, which this paper rejects, so the comparison is not like-for-like [churilov2026range, p. 5].
- **[V]** Language direction disagrees: JavaScript above Python here [spracklen2024we, p. 7], Python above JavaScript in krishna2025importing and churilov2026range [krishna2025importing, p. 8; churilov2026range, p. 7] (contradiction C1).
- **[V]** Prompt-source ordering reverses on the same corpus: Stack Overflow higher here [spracklen2024we, p. 20], LLM-written prompts higher in churilov2026range [churilov2026range, p. 7] (C2).
- **[V]** The temperature and RAG effects do not hold in Rust [zheng2026when, p. 6; zheng2026when, p. 8] (C4, C6).
- **[V]** Shared names across models also appear in zhao2025hfuzzer [zhao2025hfuzzer, p. 7].

The Range Shrinks, the Threat Remains: Re-evaluating LLM Package Hallucinations on the 2026 Frontier-Model Cohort

> **Takeaway:** Re-running Spracklen's public prompts on five 2026 frontier models gives much lower rates, about 4–5% of package mentions. But every model still invents names, 104 names are invented by all five, and about 1% of invented names were registered by someone within six months. It is a single-author preprint, yet its live registry check is the most careful measurement protocol in our set.

Item	Detail
Authors	Aleksandr Churilov (independent researcher)
Year · Venue	2026 · arXiv
Status	Preprint (not peer-reviewed). Our copy is version 4 (November 2026), which replaces all numbers in versions 1–3 and in a SCORED '26 workshop version
Identifier	DOI 10.48550/arxiv.2605.17062 (arXiv 2605.17062)
Paper type	Replication study with a preregistered live-registry re-check and a natural experiment on registrations
Relevance to us	High — RQ1, RQ2, RQ4, RQ5

In plain words

Do the newest AI models still make up package names? A package is a ready-made code library downloaded from a public registry such as PyPI (Python) or npm (JavaScript). The author took the prompt set released by Spracklen et al. and ran it unchanged on Claude Sonnet 4.6, Claude Haiku 4.5, GPT-5.4-mini, Gemini 2.5 Pro and DeepSeek V3.2 [churilov2026range, p. 4]. Every suggested name was then checked against the live registries. The paper also asks whether invented names actually get registered, and whether publishing a list of them speeds that up [churilov2026range, p. 1; churilov2026range, p. 2].

How they did it

1. Generate. The 39,969 prompts were sent to each model between 22 and 28 April 2026 [churilov2026range, p. 4].
2. Extract names from pip install commands, npm install commands inside code blocks, and import statements read with proper code parsers [churilov2026range, p. 5].
3. Clean the list. Standard-library modules, names that break registry naming rules, and import names that pipreqs (a tool that maps import names to package names) links to a real package were removed [churilov2026range, p. 5].
4. Check live. Each registry was asked in two ways, in two rounds at least 12 hours apart. A name counts as missing only if all four answers say "absent". Package creation time

was compared with generation time, and OSV (a public database of known malicious and vulnerable packages) was queried [churilov2026range, p. 5].

5. Plan in advance. The analysis plan was preregistered (written down before looking at the data), with a log of 20 deviations [churilov2026range, p. 6].
6. Test properly. Rates come with Wilson confidence intervals (a range likely to hold the true rate), and model comparisons use chi-square tests with Holm correction for multiple comparisons plus permutation tests [churilov2026range, p. 6].
7. Natural experiment. A list of shared names had been exposed publicly by accident. Registration of those names was compared with similar unpublished names [churilov2026range, p. 11; churilov2026range, p. 12].

Data and setup

- 199,840 generations in total [churilov2026range, p. 4]. 2,282 unique hallucinated names: 1,746 on PyPI and 536 on npm [churilov2026range, p. 11].
- Refusal rates range from 1.27% to 31.51% [churilov2026range, p. 4]. Temperatures were recorded but not printed [churilov2026range, p. 4].
- Estimated API cost $1,160.29 [churilov2026range, p. 17].
- At most one request per second per registry host [churilov2026range, p. 5].
- Code and aggregate results are public on GitHub; raw responses are available on request [churilov2026range, p. 15].
- Self-funded apart from a Google Cloud credit grant. The author used Claude to write analysis code and draft the text [churilov2026range, p. 15].

Key findings

- **[V]** Rates range from 3.63% (Haiku 4.5) to 5.02% (GPT-5.4-mini) of package mentions, and from 2.15% to 3.39% of generations [churilov2026range, p. 6; churilov2026range, p. 7].
- **[V]** Python is higher than JavaScript for every model, by 2.34 to 3.47 percentage points [churilov2026range, p. 7].
- **[V]** LLM-written prompts give higher rates than Stack Overflow prompts: pooled 4.79% and 5.44% against 2.28% and 3.10% [churilov2026range, p. 7].
- **[V]** 104 names (88 PyPI, 16 npm) are invented by all five models; 54 of them are one character away from a registered package [churilov2026range, p. 9]. The mean overlap between models is modest (Jaccard 0.219) [churilov2026range, p. 8].
- **[V]** The live re-check lowered every model's rate by about a fifth. The biggest correction was import names of real packages, 144–207 mentions per model [churilov2026range, p. 10].
- **[V]** 25 of 2,282 invented names (1.1%) were registered within about six months; one npm placeholder carried a malicious-package record [churilov2026range, p. 12].
- **[V]** Publishing names made no detectable difference (0 of 87 published vs 2 of 101 unpublished, p = 0.50), but the test can only detect large effects [churilov2026range, p. 12].
- **[V]** At least 23 suggested names had previously hosted malware that the registries later removed [churilov2026range, p. 12].

Limitations and weaknesses

- **[A]** Smaller, fine-tuned and quantized local models, and agentic setups, are out of scope [churilov2026range, p. 14].
- **[A]** The prompts may already be in 2026 training data, and no new held-out prompts were used [churilov2026range, p. 14].
- **[A]** The pipreqs map covers only about 1,157 import names, and some install forms are missed [churilov2026range, p. 14].
- **[I]** The paper says it follows Spracklen's extraction [churilov2026range, p. 5], but it parses imports, which Spracklen rejects [spracklen2024we, p. 6]. Its "level with 2024's best model" compares a pooled 3.63% with Spracklen's Python-only 3.59% [spracklen2024we, p. 20]; its own Python-only rate for Haiku is 4.38% [churilov2026range, p. 7].
- **[I]** One sample per prompt, so persistence is not measured.
- **[I]** It is the fourth version of a fast-revised single-author paper, and several earlier numbers could not be reproduced [churilov2026range, p. 10].

Gaps and future work this paper leaves open

- **[A]** G1: agentic setups are named as untested [churilov2026range, p. 14].
- **[A]** G4: quantized local models are out of scope [churilov2026range, p. 14].
- **[I]** G3: this is the most complete protocol in our set, but its import map is small [churilov2026range, p. 14] and it was never checked against human labels.
- **[A]** G5: the author proposes monitoring registrations [churilov2026range, p. 15]; the six-month window and low power leave long-term weaponisation open [churilov2026range, p. 12].
- **[I]** G2: registered names and names with a malware history [churilov2026range, p. 12] show why a plain existence check is not enough.

Why it matters for your research

This is the protocol to copy: two lookups per registry, two rounds, a creation-date check, OSV lookups and confidence intervals, all runnable on a laptop. It also gives the best current base rate for frontier API models. Cite it as a preprint, report per-language and pooled rates separately, and split results by prompt source, because that factor flipped since 2024.

Connections to other papers

- **[V]** Same prompts as spracklen2024we, but two results reverse: language direction [spracklen2024we, p. 7] (contradiction C1) and prompt source [spracklen2024we, p. 20] (C2).
- **[V]** Registration evidence is compatible with okwor2026hallucination: 0 of 1,641 private names after 4 days and 22 of 149 public names, none malicious [okwor2026hallucination, p. 5] (C7).
- **[V]** Python above JavaScript agrees with krishna2025importing [krishna2025importing, p. 8].
- **[V]** Import-name confusion also appears in zhao2025hfuzzer, where 34 of 46 GPT-4o hallucinations are wrong install names for correct imports [zhao2025hfuzzer, p. 9].

From Hallucination to Registration: Do the Package Names That LLMs Invent Actually Get Claimed?

> **Takeaway:** This paper asks whether invented package names actually get registered. Fresh names from five local models were all still free four days later. Of 149 names already circulating publicly, 22 were registered, mostly as harmless placeholders, and none was malicious. It is a single-author Zenodo preprint with a very short time window, so use it only to support stronger evidence.

Item	Detail
Authors	Daniel Okwor (Orygn LLC)
Year · Venue	2026 · Zenodo
Status	Preprint (Zenodo self-deposit, not peer-reviewed, single author). We treat it as grey literature (tier 4)
Identifier	DOI 10.5281/zenodo.21199426 (concept DOI; the PDF prints the version DOI 10.5281/zenodo.21199427)
Paper type	Measurement study of registration status, with content triage and an offline install demo
Relevance to us	Medium — RQ2, RQ4, RQ5

In plain words

A package hallucination is when an AI model suggests a code library that does not exist in a public registry such as PyPI (Python) or npm (JavaScript). Earlier studies count how often this happens; the author calls that exposure. This paper looks at the next step, harm: are the invented names registered, by whom, and with what intent [okwor2026hallucination, p. 1; okwor2026hallucination, p. 2]? It produces its own names with small open models run locally through Ollama, a free program for running models on your own computer [okwor2026hallucination, p. 3]. It also inspects a public list of previously reported names.

How they did it

1. Generate. Five local models (qwen2.5-coder:7b, deepseek-coder:6.7b, codellama:13b, codestral:22b, gpt-oss:20b) wrote Python and JavaScript solutions at temperatures 0.7 and 1.0 [okwor2026hallucination, p. 3].
2. Extract. Code was parsed, import names were mapped to package names, names were normalised with PEP 503 (PyPI's rule for treating spelling variants as one name), and standard-library modules were dropped [okwor2026hallucination, p. 3].
3. Two checks. A name counts as invented only if it is missing from a frozen registry snapshot taken on 30 June 2026 and the live registry also returns "not found" [okwor2026hallucination, p. 3].

4. Validate. The check was tested on 17 real and 8 fake names and classified all correctly [okwor2026hallucination, p. 4].
5. Inspect public names. 149 names from the Trend Micro and DepScope datasets were checked for registration date and, by reading setup and manifest files only, sorted into malicious, defensive placeholder, organic (an unrelated real package) or individual reservation [okwor2026hallucination, p. 4].
6. Demonstrate the payload route. A harmless test package was installed in an offline lab to show code running at install time [okwor2026hallucination, p. 7].

Data and setup

- 6,800 generations, giving 1,641 confirmed invented names [okwor2026hallucination, p. 4].
- All names were re-checked live on 4 July 2026, four days after the snapshot [okwor2026hallucination, p. 5].
- No hallucination rate is reported. The task set, prompts, seeds, quantization level and hardware are not reported.
- Code is promised for later deposit and the raw name list is withheld [okwor2026hallucination, p. 8].
- No funding or conflict-of-interest statement, despite the company affiliation [okwor2026hallucination, p. 1].

Key findings

- **[V]** No saturation: new names kept appearing at almost the same pace (0.228 against 0.247 new names per generation), and 450 of 455 new names came from a single model [okwor2026hallucination, p. 4].
- **[V]** None of the 1,641 self-generated names was registered after four days. The rule of three (if zero of n cases occur, the true rate is below about 3/n with 95% confidence) bounds this below 0.2% [okwor2026hallucination, p. 5].
- **[V]** 22 of 149 public names were registered: 19 defensive placeholders, 1 organic collision, 1 individual reservation, 1 typosquat from 2019, and 0 malicious (upper bound about 14%) [okwor2026hallucination, p. 5; okwor2026hallucination, p. 6].
- **[V]** All 19 placeholders carry the summary "Benign slopsquatting research package" [okwor2026hallucination, p. 5]. **[A]** The author attributes them to one anonymous operator, using registry metadata that is withheld [okwor2026hallucination, p. 6].
- **[V]** Code ran at install time under pip (via setup.py) and npm (via a postinstall script), before the package was ever imported [okwor2026hallucination, p. 7].
- **[A]** The public npm name mirror is unreliable [okwor2026hallucination, p. 3]. **[V]** This is partly supported by 11 older names the mirror missed [okwor2026hallucination, p. 7].
- **[A]** Claims from grey sources, such as more than 30,000 downloads of one registered invented name, are repeated but not verified here [okwor2026hallucination, p. 2].

Limitations and weaknesses

- **[A]** "Zero malicious" is not zero risk: a placeholder can be turned malicious in a later release [okwor2026hallucination, p. 8].
- **[A]** The registered sample is small and biased towards names claimed by defenders, and one person assigned the labels [okwor2026hallucination, p. 6; okwor2026hallucination, p. 8].
- **[A]** Names come from self-generated output, not real production traffic [okwor2026hallucination, p. 8].
- **[I]** The 0 of 1,641 result was close to certain by design: the names were private and checked after only four days [okwor2026hallucination, p. 5].
- **[I]** Reading only setup and manifest files can miss malicious code inside modules or wheels [okwor2026hallucination, p. 4].
- **[I]** Cited figures from other papers are misreported. Spracklen's 21.7% is described as a share "of generations" [okwor2026hallucination, p. 1], but Spracklen counts per package [spracklen2024we, p. 7]. Churilov's numbers are quoted from superseded versions [churilov2026range, p. 10].

Gaps and future work this paper leaves open

- **[A]** G1: the author proposes moving from self-generated output to live coding-agent traffic [okwor2026hallucination, p. 8].
- **[A]** G5: the author proposes to keep watching the 1,641 names [okwor2026hallucination, p. 8]; a four-day window cannot show long-term weaponisation.
- **[I]** G2: registered placeholders pass a plain existence check [okwor2026hallucination, p. 5], and could be weaponised later [okwor2026hallucination, p. 8]. A useful guard must handle them.
- **[I]** G4: it runs Ollama models but reports neither a rate nor a quantization level [okwor2026hallucination, p. 3].
- **[I]** G3: its two-check gate and the npm-mirror warning are reusable pieces of a shared protocol.

Why it matters for your research

This is the closest template for a laptop-scale pipeline: local Ollama models, a frozen snapshot as a fast first filter, and a live registry check to confirm. Use the live registry, not the npm mirror, as ground truth. Treat its registration results as weak support for the view that weaponised slopsquatting is rare so far, not as proof.

Connections to other papers

- **[V]** Compatible with churilov2026range, where 25 of 2,282 names were registered within about six months, one with malware [churilov2026range, p. 12] (C7).
- **[V]** Mostly single-model names agree with spracklen2024we, where 81% of distinct names came from one model [spracklen2024we, p. 10].
- **[V]** The install-time demo matches older malware data: 56% of real malicious packages run at install time [ohm2020backstabbers, p. 11].

Importing Phantoms: Measuring LLM Package Hallucination Vulnerabilities

> **Takeaway:** Eleven models were tested in Python, JavaScript and Rust with the open-source probing tool garak. Every model invented packages, and larger or better-coding models tended to invent fewer. The paper is useful for its date-aware definition of "hallucinated" and its proposed defence, but its statistics are weaker than they look and the defence was never tested.

Item	Detail
Authors	Arjun Krishna, Erick Galinkin, Leon Derczynski, Jeffrey Martin (University of Waterloo; NVIDIA)
Year · Venue	2025 · arXiv
Status	Preprint (not peer-reviewed). A leftover ICML 2024 footer on page 1 does not show a real venue; we treat the venue as unverified
Identifier	DOI 10.48550/arxiv.2501.19012 (arXiv 2501.19012)
Paper type	Measurement study with correlation analysis
Relevance to us	High — RQ1, RQ3, RQ4, RQ5

In plain words

A package hallucination is when an AI model writes code that uses a library that does not exist in the public registry (npm for JavaScript, PyPI for Python, crates.io for Rust). The authors ask how often this happens and whether it depends on the language, model size, or whether the model is specialised for code [krishna2025importing, p. 2]. Their new idea is a date-aware definition. A package also counts as hallucinated if it was registered after the model's knowledge cutoff, the date its training data ends [krishna2025importing, p. 3]. The model cannot have known such a package, so a match is suspicious and may be a squatter.

How they did it

1. Build a prompt grid: 7 request stubs combined with 13 coding tasks for each language, each request repeated five times [krishna2025importing, p. 4; krishna2025importing, p. 12].
2. Send prompts through garak and extract imported names with regular expressions (simple text patterns) [krishna2025importing, p. 3; krishna2025importing, p. 12].
3. Compare names with registry lists and release dates collected on 30 January 2025 [krishna2025importing, p. 4].
4. Label a name hallucinated if it is unregistered, registered after the cutoff, or (when the cutoff is unknown) registered less than 90 days before the model's release [krishna2025importing, p. 3].

5. Compute the Package Hallucination Rate (PHR): the share of prompts with at least one hallucinated package [krishna2025importing, p. 4].
6. Correlate PHR with model size and with HumanEval and MBPP coding benchmark scores, and compare code-specialised with general models using a t-test [krishna2025importing, p. 6; krishna2025importing, p. 7].

Data and setup

- 11 models, including GPT-4o, Llama-3.1 8B and 70B, Qwen2.5-Coder 7B, StarCoder2-15B and Llama-3.1-Nemotron-70B [krishna2025importing, p. 3].
- Registry sizes: npm 3,391,235, PyPI 604,814 and crates.io 169,823 packages [krishna2025importing, p. 4].
- The number of generations is not reported. **[I]** All 33 published rates are exact multiples of 1/455, which points to 455 prompts per model-language cell [krishna2025importing, p. 4].
- Temperature, hardware, quantization and how cutoffs were found are not reported. No code link is given [krishna2025importing, p. 10].
- Three of the four authors work at NVIDIA [krishna2025importing, p. 1].

Key findings

- **[V]** Every model hallucinated in every language. PHR ranged from 0.22% (Nemotron-70B, JavaScript and Rust) to 46.15% (Granite-3.0, Python) [krishna2025importing, p. 4]. **[I]** 0.22% is a single prompt out of 455.
- **[V]** Mean PHR: JavaScript 14.73%, Python 23.14%, Rust 24.74% [krishna2025importing, p. 8].
- **[V]** GPT-4o: 1.76% in JavaScript, 3.52% in Python, 10.99% in Rust [krishna2025importing, p. 4].
- **[V]** Larger models had lower PHR ($r = -0.542$, $p = 0.00114$) [krishna2025importing, p. 6]. **[I]** This treats 33 model-language points as independent; with 11 models the p-value would be about 0.085, and GPT-4o's size is a 200B estimate [krishna2025importing, p. 3].
- **[V]** Higher HumanEval scores went with lower PHR ($r = -0.7887$ over 10 models, no p-value) [krishna2025importing, p. 7].
- **[V]** Code-specialised models had higher average PHR, but the difference was significant only for JavaScript and disappeared without GPT-4o. **[A]** The authors conclude there is likely no meaningful difference [krishna2025importing, p. 6].
- **[A]** Leading ("induced") prompts gave nearly double the natural rate; no figures are given [krishna2025importing, p. 5].
- **[A]** Proposed defence: a date-aware list of packages published before each model's cutoff. It is not evaluated [krishna2025importing, p. 8].

Limitations and weaknesses

- **[A]** Eleven models may be too few to show a difference between model types [krishna2025importing, p. 6].

- **[A]** The proposed list cannot catch malicious packages that are already old [krishna2025importing, p. 8].
- **[I]** There is no mapping from import name to package name and no standard-library filter [krishna2025importing, p. 12]. Real imports such as sklearn or cv2 may therefore count as hallucinations. In churilov2026range such import names were the largest false-positive class [churilov2026range, p. 10].
- **[I]** The text contradicts the paper's own tables, for example on Mamba-Codestral [krishna2025importing, p. 6; krishna2025importing, p. 7].
- **[I]** The abstract says defences were developed and evaluated [krishna2025importing, p. 1], but the defence is only proposed [krishna2025importing, p. 8].
- **[I]** The 13 tasks are unusual and include possibly fictional targets such as "StrombergDB", and some prompt stubs use unexplained personas [krishna2025importing, p. 12].
- **[I]** NVIDIA authors evaluate NVIDIA models, and an NVIDIA model ranks best, with no conflict-of-interest statement [krishna2025importing, p. 1; krishna2025importing, p. 4].

Gaps and future work this paper leaves open

- **[I]** G2: the date-aware list is proposed but untested [krishna2025importing, p. 8]. It is a natural candidate rule for a guard benchmark.
- **[I]** G3: its per-prompt unit, regex extraction and date rule differ from other papers, a clear example of why rates cannot be pooled.
- **[I]** G1: only single chat responses were studied; no agent ever installed anything.
- **[A]** The authors call for defences that work across languages and for testing more models [krishna2025importing, p. 6].

Why it matters for your research

Reuse the date-aware definition: this is one of the few papers in our set that handle packages registered after a model's cutoff. The setup (garak and 7–8B models) is laptop-friendly [krishna2025importing, p. 3]. Do not cite the size correlation as strong evidence, and do not compare its per-prompt rates with per-mention rates from other papers. If you build a guard, implementing its untested verification list is a cheap, novel step.

Connections to other papers

- **[V]** Python above JavaScript agrees with churilov2026range [churilov2026range, p. 7] and disagrees with spracklen2024we [spracklen2024we, p. 7] (contradiction C1).
- **[V]** Its size trend conflicts with Rust results showing no size effect [zheng2026when, p. 6] and with a newer Llama model producing more invented names in zhao2025hfuzzer [zhao2025hfuzzer, p. 7] (C3).
- **[V]** zhao2025hfuzzer adopts this paper's PHR definition [zhao2025hfuzzer, p. 5] and also uses a before-cutoff rule [zhao2025hfuzzer, p. 4].
- **[A]** zhao2025hfuzzer argues that import-based extraction, as used here, misses errors made at the install step [zhao2025hfuzzer, p. 9].

Secure or Suspect? Investigating Package Hallucinations of Shell Command in Original and Quantized LLMs

> **Takeaway:** This is the only study in our set that directly varies quantization, the compression that lets models run on ordinary hardware. For small Qwen models writing Go install commands, 4-bit versions invented package paths almost every time. But the effect is not uniform, and the measurement counts file paths as "packages", so treat it as weak evidence.

Item	Detail
Authors	Md Nazmul Haque, Elizabeth Lin, Lawrence Arkoh, Biruk Tadesse, Bowen Xu
Year · Venue	2025 · arXiv
Status	Preprint (not peer-reviewed). The PDF still carries conference-template residue, suggesting an early draft
Identifier	DOI 10.48550/arxiv.2512.08213 (arXiv 2512.08213)
Paper type	Empirical measurement study (5 model sizes × 3 precision levels × 3 prompt sets)
Relevance to us	High — RQ1, RQ2, RQ4, RQ5

In plain words

Most people who run AI models on a laptop use quantized versions. Quantization stores the model's numbers with fewer bits (for example 8 or 4 instead of 16 or 32), so the model needs less memory but may become less accurate. This paper asks whether quantized models invent more package names when they write shell commands that install Go packages [haque2025secure, p. 2]. In Go, packages are named by web addresses such as a GitHub repository path and installed with go get. An attacker could squat an invented path by creating a free GitHub account and repository with that name [haque2025secure, p. 2; haque2025secure, p. 7].

How they did it

1. Build three prompt sets with GPT-4o: Stack Overflow Go questions rewritten as tasks, MBPP programming tasks adapted to Go, and paraphrased descriptions from the awesome-go package list [haque2025secure, p. 4].
2. Run Qwen models of five sizes (0.5B to 14B) at full precision, 8-bit and 4-bit, using the bitsandbytes library [haque2025secure, p. 4; haque2025secure, p. 5].
3. Take the package paths from go get or go install commands and request each address over the network. An unreachable address counts as hallucinated [haque2025secure, p. 4].
4. Check reachable packages for known vulnerabilities in OSV, a public vulnerability database [haque2025secure, p. 4].

5. Report the Package Hallucination Rate (hallucinated strings divided by all generated package strings) and the Vulnerable Package Rate [haque2025secure, p. 4; haque2025secure, p. 5].

Data and setup

- 5,435 Stack Overflow, 974 MBPP and 1,746 awesome-go prompts, 8,155 in total, each run on 15 model configurations [haque2025secure, p. 4].
- The number of package strings per condition ranges from 0 to 21,439 [haque2025secure, p. 6].
- Run on one A100 GPU [haque2025secure, p. 5].
- Exact model versions, prompt wording, temperature, the extraction method and the date of the checks are not reported. Each prompt appears to have been run once, with no statistical tests.
- A replication package is offered at an anonymous link [haque2025secure, p. 2].

Key findings

- **[V]** At 4-bit, the 0.5B model invented 96.11%, 98.78% and 96.66% of package strings on the three prompt sets, against 41–54% at full precision [haque2025secure, p. 6].
- **[V]** The 3B model at 4-bit reached 92.09%, 97.60% and 87.74% [haque2025secure, p. 6].
- **[V]** Exception: the 7B model on Stack Overflow prompts fell from 34.00% to 10.21% at 4-bit, while producing far fewer package strings [haque2025secure, p. 5].
- **[A]** The authors call the 8-bit effect a "moderate and consistent increase" [haque2025secure, p. 5]. **[V]** Their own table shows several cells falling at 8-bit and one rising sharply, 48.39% to 69.59% for 7B on MBPP [haque2025secure, p. 6].
- **[V]** Some rates rest on tiny counts: the 3B model produced 3 and 1 package strings on Stack Overflow at full and 8-bit precision, and none on MBPP [haque2025secure, p. 6].
- **[V]** In the one breakdown table, 2,333 of 4,751 "hallucinated" strings (49%) are file paths or free-form text, not installable package paths [haque2025secure, p. 8].
- **[V]** The quantization effect on known-vulnerable packages is mixed: up from 12.89% to 19.58% for 7B on Stack Overflow, down in other cells [haque2025secure, p. 5; haque2025secure, p. 6].
- **[A]** The authors recommend 8-bit for constrained settings and "extreme caution" with 4-bit [haque2025secure, p. 7].

Limitations and weaknesses

- **[A]** Existence was checked by web reachability for simplicity; an unreachable address might still be a valid Go package, and the authors suggest running go get instead [haque2025secure, p. 7].
- **[A]** Prompts were not varied [haque2025secure, p. 7].
- **[I]** The summary percentages in the discussion use only the Stack Overflow set [haque2025secure, p. 7], and the "consistent increase" claim goes beyond the data.

- **[I]** Counting file paths and free text as packages [haque2025secure, p. 8] overstates the number of squattable names.
- **[I]** One model family and one quantization library. Laptop runtimes such as Ollama use a different format (GGUF), so the results may not transfer.
- **[I]** GPT-4o kept only tasks it answered with install commands [haque2025secure, p. 4], which biases the prompts towards install-heavy tasks.
- **[I]** No repeated runs, so persistence of names is not measured, and whether invented GitHub paths could actually be registered was not checked.

Gaps and future work this paper leaves open

- **[I]** G4: this is the only quantization study, and it covers Go only with bitsandbytes [haque2025secure, p. 6]. The effect of the quantization levels shipped by local runtimes on PyPI and npm is untested.
- **[A]** G1: the authors call for verifying packages by actually running go get [haque2025secure, p. 7]; no study yet records real installs.
- **[I]** G3: its reachability check and counting of non-package strings are a clear example of why the field needs a shared measurement rule.
- **[I]** G2: the setting suggests a simple guard: check every install target against the registry or module proxy before running it.

Why it matters for your research

It is the main motivation for G4, because quantized small models are exactly what a laptop runs. It is also a list of pitfalls to avoid: count only installable names, report every denominator, run each prompt several times, and use a proper registry lookup. Quote its 4-bit numbers only with the caveats above.

Connections to other papers

- **[V]** spracklen2024we ran all open models in quantized form [spracklen2024we, p. 6], so model size and quantization are mixed up in earlier results (contradiction C3).
- **[A]** churilov2026range explicitly leaves quantized local models out of scope [churilov2026range, p. 14].
- **[V]** okwor2026hallucination runs local models through Ollama but reports no hallucination rate [okwor2026hallucination, p. 3].
- **[V]** Go adds to other ecosystems studied elsewhere, such as Rust [zheng2026when, p. 5].

When LLMs Invent Rust Crates: An Empirical Study of Hallucination Patterns and Mitigation

> **Takeaway:** In Rust code written by six AI models, about one in five references to a crate (a Rust package) pointed to a crate that does not exist, and model size and temperature barely changed this [zheng2026when, p. 5; zheng2026when, p. 6]. Almost half of these "invented crates" are mix-ups with Rust's own standard library, and part of the count may come from the authors' name parser, so treat the headline rate with care.

Item	Detail
Authors	Jieming Zheng, Hao Guan, Yepang Liu
Year · Venue	2026 · arXiv
Status	Preprint (not peer-reviewed); a reported conference acceptance is not verified
Identifier	arXiv 2606.08444 (DOI 10.48550/arxiv.2606.08444)
Paper type	Measurement study plus a prompt-based mitigation experiment
Relevance to us	High — RQ1, RQ2, RQ3, RQ4, RQ5

In plain words

Rust is a programming language. Its packages are called "crates" and are published on the registry crates.io. A package hallucination happens when an AI model names a package that does not exist. An attacker can register such an invented name and fill it with malware; this attack is called slopsquatting. Earlier studies looked at Python and JavaScript; the authors present this as the first large study for Rust [zheng2026when, p. 1]. One caveat matters for us: in Rust, mentioning a crate in a source file does not download it [zheng2026when, p. 1]. So this paper measures names in generated text, not what actually gets installed.

How they did it

1. They built 2,794 Rust coding tasks from three sources: 1,001 Stack Overflow questions, 793 tasks made from documented functions in popular GitHub projects, and 1,000 tasks that a Qwen model wrote from crate descriptions [zheng2026when, p. 2; zheng2026when, p. 3].
2. Each model wrote code for every task with its default settings [zheng2026when, p. 4].
3. A script pulled crate names out of the code with three text patterns: use lines, path prefixes such as name::item, and extern crate lines [zheng2026when, p. 4].
4. Each name was compared with a reference list: every crates.io name, plus Rust's standard and compiler crates and known aliases. Any name not on the list counted as hallucinated [zheng2026when, p. 4; zheng2026when, p. 5].
5. The main measure is the Crate Hallucination Rate (CHR): hallucinated references divided by all extracted references [zheng2026when, p. 5].

6. To describe patterns they used edit distance (how many single-letter changes turn one name into another) and the overlap of invented names between models [zheng2026when, p. 7].
7. They tested two prompt-based fixes on the three open models. Self-refinement asks the model to check its own crate names and regenerate up to 3 times. RAG (retrieval-augmented generation) adds descriptions of 5 matching crates, drawn from the 10,000 most-downloaded, to the prompt [zheng2026when, p. 8].

Data and setup

- 14 model variants from six families: GPT-5, Gemini 2.5 Pro, Claude 4 Opus, Qwen2.5-Coder (0.5B to 32B), DeepSeek-Coder-V2 (16B and 236B) and OpenCoder (1.5B and 8B). "B" means billions of parameters, a rough measure of model size [zheng2026when, p. 4].
- Main run: 16,764 code snippets with 48,494 extracted crate references [zheng2026when, p. 5]. **[I]** That is one answer per task per model.
- Temperature (a setting that makes output more random) was varied from 0 to 2 on four models [zheng2026when, p. 6].
- Open models were run through Ollama, a tool for running models locally, on a server with 8 RTX 6000 Ada GPUs [zheng2026when, p. 5].
- Not reported: the quantization level of the Ollama models and the date of the crates.io name list.
- Tasks, prompts, scripts and results are shared on figshare [zheng2026when, p. 10].

Key findings

- **[V]** 10,779 of 48,494 crate references were hallucinated, an overall CHR of 22.23% [zheng2026when, p. 5].
- **[V]** Gemini 2.5 Pro had the lowest rate (16.18%) and Claude 4 Opus the highest (26.90%) [zheng2026when, p. 5].
- **[V]** Size did not predict the rate within Qwen2.5-Coder ($p = 0.912$). Temperature had no significant effect ($p = 0.4859$), while the choice of model did ($p = 0.0173$) [zheng2026when, p. 5; zheng2026when, p. 6].
- **[V]** 45.47% of hallucinations are standard-library or internal modules treated as outside crates, led by thread and duration [zheng2026when, p. 6].
- **[V]** The other invented names sit close to real ones: 40.78% are 1–2 edits from a real crate and 86.48% within 5; 32.96% drop a hyphen that the real crate has [zheng2026when, p. 7].
- **[V]** 38.9% of frequently invented names are borrowed from other ecosystems, such as queryparser from Python [zheng2026when, p. 8].
- **[V]** Self-refinement lowered the rate by 2.22–3.26 points. RAG lowered it by 1.92 and 0.96 points for two models and raised it by 0.34 for Qwen2.5-Coder [zheng2026when, p. 9].

Limitations and weaknesses

- **[A]** Whether invented crates would be installed, break builds or be exploitable was not measured [zheng2026when, p. 10].
- **[A]** Alias coverage is incomplete, so the true rate is "likely slightly lower" [zheng2026when, p. 4].
- **[A]** A third of the tasks were written by a Qwen model, and models change fast [zheng2026when, p. 10].
- **[I]** The path-prefix pattern may also catch type names such as Duration::from_secs. Odd top-10 names such as trefcell and tu32 look like a parser bug that the paper does not discuss [zheng2026when, p. 6].
- **[I]** The counts do not match: 10,779 hallucinated references in one section, 13,332 non-module occurrences in the next [zheng2026when, p. 5; zheng2026when, p. 7].
- **[I]** The abstract claims "surprisingly consistent" rates and no loss of code quality, but the model effect is significant and code quality was never measured [zheng2026when, p. 1; zheng2026when, p. 6].
- **[I]** The paper does not say how hyphens in crate names were matched to underscores in code, which affects the 32.96% hyphen finding.
- **[I]** Ollama's default downloads are usually quantized, which may blur size comparisons. One answer per task and no confidence intervals.

Gaps and future work this paper leaves open

- **[A]** G1: installation through Cargo, Rust's package tool, was not measured [zheng2026when, p. 10]. What models actually install remains open.
- **[A]** G2: the authors propose a Rust-aware guard that separates standard modules from outside crates, checks near-miss names against the registry and cross-checks the project's Cargo.toml file, but they do not test it [zheng2026when, p. 9].
- **[I]** G3: its unit (every source-level reference, including standard-library mix-ups) differs from other papers' units, so 22% cannot be compared directly with Python or JavaScript rates [zheng2026when, p. 4].
- **[I]** G4: the unreported quantization of the Ollama models is a hidden confound.
- **[A]** The prompt fixes still need testing on commercial models [zheng2026when, p. 9].

Why it matters for your research

This is the only Rust study in the set. Its main lesson is about measurement: the reference list and the parser decide much of the rate, so hand-check a sample of flagged names in any study of your own. The near-miss pattern (most invented names are a few letters from a real one) supports adding a lookalike check to a pre-install guard. The small Qwen2.5-Coder models it used run on a laptop through Ollama, and its data are public, so a partial replication is cheap.

Connections to other papers

- **[V]** Temperature: spracklen2024we found rates rising sharply above temperature 1 [spracklen2024we, p. 8]; here there was no effect for Rust (evidence matrix contradiction 4).

- **[V]** Size: krishna2025importing found that larger models hallucinate less [krishna2025importing, p. 6]; no size effect here (contradiction 3).
- **[V]** RAG cut Python rates sharply in spracklen2024we [spracklen2024we, p. 13] but helped little here (contradiction 6).
- **[I]** spracklen2024we reported that 48.6% of invented Python names are 6 or more edits from a real one [spracklen2024we, p. 11]. This paper counts occurrences rather than names, so the two figures are not directly comparable.
- **[V]** When the fake name is planted in the prompt, dasu2026refusing finds nonexistent Rust crates fabricated on 87% of prompts [dasu2026refusing, p. 8]. That measures accepting a name, not inventing one.

LLM Hallucinations in Practical Code Generation: Phenomena, Mechanism, and Mitigation

> **Takeaway:** In realistic Python tasks set inside real projects, most AI coding mistakes come from misunderstanding the task, and problems with missing packages make up under 1% of hallucinations [zhang2025llm, p. 4; zhang2025llm, p. 6]. Read it as background: it maps the kinds of code hallucination, but it does not measure invented PyPI packages.

Item	Detail
Authors	Ziyao Zhang, Chong Wang, Yanlin Wang, Ensheng Shi, Yuchi Ma, Wanjun Zhong, Jiachi Chen, Mingzhi Mao, Zibin Zheng (published version; our arXiv copy lists five authors)
Year · Venue	2025 · Proceedings of the ACM on Software Engineering (PACMSE), ISSTA 2025 issue
Status	Peer-reviewed (PACMSE / ISSTA 2025); our copy is the arXiv version
Identifier	DOI 10.1145/3728894 (arXiv 2409.20550)
Paper type	Empirical study: hand-built taxonomy of errors plus a small mitigation pilot
Relevance to us	Low — RQ1 (marginal), RQ3 and RQ4 (background)

In plain words

Many studies test AI coding on small, self-contained functions. This paper uses repository-level tasks instead: the model must write a function that fits into a real project with its own files and dependencies [zhang2025llm, p. 1]. The authors read thousands of generated snippets and sorted the mistakes into a taxonomy (a structured list of error types) [zhang2025llm, p. 3; zhang2025llm, p. 4]. Package hallucination is only one small corner of that map. The paper is useful to us mainly for context and for a warning about definitions.

How they did it

1. They used CoderEval Python, a benchmark of 230 tasks taken from real projects, each with tests and a working project environment [zhang2025llm, p. 3].
2. Six models produced 10 answers per task with nucleus sampling at temperature 0.6, a moderately random setting [zhang2025llm, p. 3].
3. Two authors open-coded (read examples and created labels from what they saw) 1,380 snippets from 23 tasks to build the first version of the taxonomy [zhang2025llm, p. 3].
4. Three volunteers labelled the remaining snippets, and one author reviewed all labels [zhang2025llm, p. 3; zhang2025llm, p. 10].
5. They compared models and proposed root causes [zhang2025llm, p. 8].

6. A pilot fix used RAG (retrieval-augmented generation): it added the 10 most similar code snippets from the same project to the prompt. Success was measured by Pass@1, the share of tasks where the first answer passes the tests [zhang2025llm, p. 9].

Data and setup

- Models: GPT-3.5-Turbo, CodeGen-350M, PanGu-alpha-2.6B, DeepSeekCoder-6.7B, CodeLlama-7B-Python and StarCoder2-7B [zhang2025llm, p. 3].
- **[I]** 230 tasks × 6 models × 10 answers gives 13,800 snippets by our arithmetic; the paper does not state the total number of hallucinations.
- A "non-existent package" means one that is not available in the task's environment [zhang2025llm, p. 6]. There is no check against PyPI.
- Code, data and results are on GitHub [zhang2025llm, p. 1]. Hardware and exact prompts are not reported.

Key findings

- **[V]** Three categories: task requirement conflicts 43.53%, factual knowledge conflicts 31.91% and project context conflicts 24.56% of hallucinations [zhang2025llm, p. 4; zhang2025llm, p. 5; zhang2025llm, p. 6].
- **[V]** Environment conflicts, which include importing a package missing from the environment, are only 0.94%. The example is a package called _lfu_cache [zhang2025llm, p. 6].
- **[I]** _lfu_cache is not a valid PyPI distribution name, so it is probably a missing internal module. The 0.94% is therefore an upper bound on invented-package cases.
- **[V]** API knowledge conflicts are much more common (20.41%), for example a parameter that a real function does not have, as in timedelta(year=1) [zhang2025llm, p. 6].
- **[V]** Library knowledge conflicts make up 2.68% [zhang2025llm, p. 5].
- **[V]** One security-relevant example: generated code used yaml.load instead of the safer safe_load [zhang2025llm, p. 5].
- **[V]** Task requirement conflicts are the most frequent category for all six models [zhang2025llm, p. 8].
- **[V]** RAG raised Pass@1 for every model, for example ChatGPT from 10.40% to 12.61% and CodeLlama from 2.17% to 5.22%. **[A]** The authors call the gain modest [zhang2025llm, p. 9].

Limitations and weaknesses

- **[A]** Python only and only 230 tasks [zhang2025llm, p. 10].
- **[A]** No formal inter-rater reliability measure (a statistic for how far independent labellers agree) [zhang2025llm, p. 10].
- **[I]** The fix was judged by Pass@1, not by counting hallucinations, so it does not show that RAG reduces hallucinations.
- **[I]** Without a registry check, invented PyPI names cannot be separated from real packages that simply are not installed.

- **[I]** Shares are given without totals, so per-answer rates cannot be worked out.
- **[I]** The models are old and small; some Pass@1 values are as low as 0.04% [zhang2025llm, p. 9].
- **[I]** Our arXiv copy lists five authors against nine in the published version, so the content may differ [zhang2025llm, p. 1].

Gaps and future work this paper leaves open

- **[I]** Our gap report does not use this paper as evidence for any gap (G1–G5).
- **[I]** G3: its definition ("not in the environment") is one more example of the unshared rules for deciding what counts as a hallucinated package [zhang2025llm, p. 6].
- **[A]** The authors suggest better ways to find and locate hallucinations, richer retrieval sources and multi-agent systems with tools [zhang2025llm, p. 9; zhang2025llm, p. 10].

Why it matters for your research

Cite it as related work only. It shows that package hallucination is a narrow slice of code hallucination when the model writes code inside an existing project. It also gives a definitional warning: "does not exist in the environment" is not the same as "does not exist on PyPI". If you build a measurement, record these as separate outcomes.

Connections to other papers

- **[V]** mirandapena2026empirical also judges missing modules against the installed environment rather than a registry [mirandapena2026empirical, p. 18].
- **[V]** Studies that ask models to write whole programs from a prompt find package hallucination in every model tested [spracklen2024we, p. 7; churilov2026range, p. 6]. **[I]** The different task type likely explains why it is so rare here.
- **[I]** Its API knowledge conflicts resemble the library member hallucinations (calls to functions a real library lacks) studied by twist2026library [twist2026library, p. 1].

HFuzzer: Testing Large Language Models for Package Hallucinations via Phrase-based Fuzzing

> **Takeaway:** HFuzzer is a testing tool that automatically writes many varied coding tasks to provoke package hallucinations. It triggered them in all nine models tested and found on average 2.60 times more unique invented Python packages than an adapted baseline. Its GPT-4o case study shows that most errors are wrong install names for code that imports correctly.

Item	Detail
Authors	Yukai Zhao, Menghan Wu, Xing Hu, Xin Xia
Year · Venue	2025 · 40th IEEE/ACM International Conference on Automated Software Engineering (ASE 2025)
Status	Peer-reviewed. Our copy is the arXiv v2 version (2509.23835), so page numbers follow that copy
Identifier	DOI 10.1109/ase63991.2025.00225 (arXiv 2509.23835)
Paper type	Tool proposal with a controlled comparison, ablation study and case study
Relevance to us	High — RQ1, RQ2, RQ4, RQ5

In plain words

Fuzzing means automatically generating many varied inputs to find where a system fails. Here the inputs are coding tasks and the failure is a package hallucination: the AI model suggests installing a Python library that does not exist on PyPI, the main Python package registry. One model (the "tester") writes the tasks. Another model (the "target") writes code and is then asked how to install the packages it used [zhao2025hfuzzer, p. 3; zhao2025hfuzzer, p. 4]. The authors argue that fixed prompt sets from earlier studies are too small and narrow to test models well [zhao2025hfuzzer, p. 1].

How they did it

1. Extract phrases. The tester model turns the names and descriptions of popular Python packages into short phrases (object, action, detail) stored in pools [zhao2025hfuzzer, p. 3].
2. Run rounds. Each round combines phrases, the tester writes a task, the target writes code, and the target is then asked for install commands [zhao2025hfuzzer, p. 3; zhao2025hfuzzer, p. 4].
3. Classify each package as standard library, existing before the model's knowledge cutoff, a package from another language, or non-existent [zhao2025hfuzzer, p. 4].
4. Learn as it goes. Phrases that trigger hallucinations get more weight, and successful tasks seed new ones [zhao2025hfuzzer, p. 4; zhao2025hfuzzer, p. 5].

5. Compare with GPTFuzzer-A, a jailbreak fuzzer the authors adapted, because no earlier tool tests package hallucination [zhao2025hfuzzer, p. 5].
6. Remove parts one at a time (an ablation study) to see what each contributes [zhao2025hfuzzer, p. 8], and measure task variety with a Diversity Index based on clustering text embeddings [zhao2025hfuzzer, p. 5].

Data and setup

- Seeds: the top 100 Python packages on Libraries.io by SourceRank; the top 1,000 for the case study [zhao2025hfuzzer, p. 6; zhao2025hfuzzer, p. 8].
- Nine models, from 7–8B open models (Llama-3, Llama-3.1, Qwen2.5-Coder, DeepSeek-Coder, Mistral-7B) to Llama-3.3-70B, DeepSeek-V3, GPT-4o mini and GPT-4o [zhao2025hfuzzer, p. 5].
- Every model was used as both tester and target: 81 pairs, 1,000 rounds each [zhao2025hfuzzer, p. 6].
- Tester temperature 0, target temperature 0.7, maximum 3,000 tokens, run on a server with four A800 GPUs [zhao2025hfuzzer, p. 6].
- Python only [zhao2025hfuzzer, p. 5]. The case study ran 10,000 rounds against GPT-4o [zhao2025hfuzzer, p. 8].
- Code is released on GitHub [zhao2025hfuzzer, p. 2]. The number of responses per model and the registry query date are not reported.

Key findings

- **[V]** HFuzzer triggered hallucinations in all nine models and found 2.60 times more unique invented packages than the baseline; over three repeated runs the gain was 3.02 times, significant for most pairs [zhao2025hfuzzer, p. 7].
- **[V]** Average hallucination rates ranged from 0.26–0.66% (GPT-4o, GPT-4o mini, DeepSeek-V3, Qwen2.5-Coder) to 16.47% (Mistral-7B) [zhao2025hfuzzer, p. 6].
- **[V]** 190 non-existent and 176 other-language names were suggested by more than one model; pkg_resources was suggested by all nine [zhao2025hfuzzer, p. 7].
- **[V]** In the GPT-4o case study, 34 of 46 invented packages were "package errors" (correct import, wrong install name) and 12 were "code errors" [zhao2025hfuzzer, p. 9].
- **[V]** In 12 cases the model's own code comments marked a name as a placeholder or assumption, yet it still gave an install command [zhao2025hfuzzer, p. 9].
- **[V]** Packages released after the model's cutoff, used as seeds, triggered more hallucinations [zhao2025hfuzzer, p. 9].
- **[V]** Removing the phrase step cut the number of unique invented packages by 2.06 times [zhao2025hfuzzer, p. 8].
- **[A]** Llama-3.1 produced more unique invented names than the older Llama-3, and Qwen2.5-Coder's low rate partly reflects frequent refusals [zhao2025hfuzzer, p. 7].

Limitations and weaknesses

- **[A]** Python only; extraction by text patterns can misfire; temperature effects were tested on only two model pairs [zhao2025hfuzzer, p. 10].
- **[I]** The low rates for strong models are not base rates. The fuzzer hunts for unique names, not for a typical mix of developer prompts [zhao2025hfuzzer, p. 6].
- **[I]** Counting pkg_resources as a hallucination is debatable, because it is an import name shipped with the setuptools package [zhao2025hfuzzer, p. 7].
- **[I]** The baseline was adapted by the authors themselves [zhao2025hfuzzer, p. 5], repeated runs used only three runs and three targets [zhao2025hfuzzer, p. 7], and case-study labels have no agreement statistic [zhao2025hfuzzer, p. 9].
- **[I]** How release dates were obtained is not reported; the registry date can only be approximated from a reference accessed in March 2025 [zhao2025hfuzzer, p. 12].
- **[A]** A GPT-4o invented name appearing in GitHub repositories is reported without counts [zhao2025hfuzzer, p. 10].

Gaps and future work this paper leaves open

- **[I]** G1: models give install commands even for names they mark as placeholders [zhao2025hfuzzer, p. 9]. What a coding agent that really runs those commands does is untested.
- **[I]** G2: HFuzzer's found names are a ready test set for a pre-install guard, and the placeholder cases suggest a comment-aware filter.
- **[I]** G3: most GPT-4o errors are install-name errors for correct imports [zhao2025hfuzzer, p. 9]. This shows why studies need a shared rule for import names versus package names.
- **[A]** The authors call for more models and languages [zhao2025hfuzzer, p. 11] and for using HFuzzer output as fine-tuning data [zhao2025hfuzzer, p. 10].

Why it matters for your research

HFuzzer is a reusable generator of hard test prompts, and its two-step protocol (write code, then ask how to install it) catches errors that reading import lines misses. The authors report that it still works with smaller tester models [zhao2025hfuzzer, p. 7], so a local 7B pair may be feasible on a laptop. Do not quote its rates as typical; quote its counts of unique names and its error types.

Connections to other papers

- **[V]** It uses the PHR definition from krishna2025importing [krishna2025importing, p. 4].
- **[I]** Its install-name finding supports spracklen2024we's choice not to parse imports [spracklen2024we, p. 6], and contrasts with churilov2026range, which does parse imports [churilov2026range, p. 5].
- **[V]** Names shared across models echo the 104 names invented by all five 2026 models in churilov2026range [churilov2026range, p. 9].
- **[V]** A newer Llama inventing more names [zhao2025hfuzzer, p. 7] contrasts with the size trend in krishna2025importing [krishna2025importing, p. 6] (contradiction C3).

Refusing the Impossible: A Taxonomy and Benchmark for Code Hallucination in Large Language Models

> **Takeaway:** When a coding request names a package that does not exist, open AI models usually write code for it instead of saying it does not exist, and bigger models barely do better on this [dasu2026refusing, p. 6; dasu2026refusing, p. 8]. The fake name is written into the prompt, so this paper measures how often models accept a planted name, not how often they invent one.

Item	Detail
Authors	Vishnu Asutosh Dasu, Ashish Kundu, Gang Tan
Year · Venue	2026 · arXiv
Status	Preprint (not peer-reviewed)
Identifier	arXiv 2609.03267 (DOI 10.48550/arxiv.2609.03267)
Paper type	Benchmark and evaluation study with an AI judge checked against human labels
Relevance to us	High — RQ1, RQ2, RQ3, RQ4, RQ5

In plain words

Most coding benchmarks contain only tasks that can be solved. This paper asks what happens when a request cannot be satisfied, for example because it names a package that does not exist or asks for an impossible algorithm. The right answer is to refuse [dasu2026refusing, p. 1]. The authors call code built on something that does not exist "code hallucination", sort these failures into a taxonomy (a structured list of types), and release a benchmark to test them [dasu2026refusing, p. 1]. Matched solvable tasks check that a model is not simply refusing everything.

How they did it

1. They hand-wrote 270 unsatisfiable requests in 24 subcategories. One family covers plausible but nonexistent things, such as packages on PyPI, npm, crates.io and Go, Java APIs and C headers [dasu2026refusing, p. 4].
2. Each request comes with a note naming the planted fake item, for example a crate called serde_ultra [dasu2026refusing, p. 4].
3. They added 91 solvable control tasks that differ only slightly from the fake ones [dasu2026refusing, p. 4].
4. Requests cover six programming languages and ten "framings", such as internal documentation or a learner's question [dasu2026refusing, p. 4].
5. 17 open-weight models (models whose weights can be downloaded) each answered once at temperature 0, the least random setting [dasu2026refusing, p. 5].

6. Claude Opus 4.6 acted as judge with a fixed rubric, labelling each answer as correct, hallucination or refusal [dasu2026refusing, p. 4].
7. The judge was checked against 138 human labels, and results carry 95% bootstrap intervals (ranges of likely values found by resampling the prompts) [dasu2026refusing, p. 5].

Data and setup

- 6,137 judged responses from 17 models with 6.7B to 671B parameters [dasu2026refusing, p. 1].
- Models up to 32B ran with 4-bit quantization (weights stored with fewer bits to save memory); larger ones ran at full precision [dasu2026refusing, p. 10].
- The package subsets are small: twelve nonexistent npm prompts and ten nonexistent crate prompts [dasu2026refusing, p. 9].
- Judge agreement with humans: 94% on outcomes, Cohen's kappa 0.91 (an agreement score corrected for chance, where 1 is perfect) [dasu2026refusing, p. 5].
- Inference software and hardware are not reported. Code and prompts are released [dasu2026refusing, p. 2].

Key findings

- **[V]** Models wrote ungrounded code for 54% of unsatisfiable requests and refused 31% [dasu2026refusing, p. 5]. Only 1 of 1,547 control answers was a wrongful refusal [dasu2026refusing, p. 2].
- **[V]** Scale helps with impossible tasks (hallucination 0.84 for small models, 0.32 for the largest) but much less with nonexistent things (0.88 to 0.61), and least with fake npm packages and Rust crates (1.00 to 0.85) [dasu2026refusing, p. 6].
- **[V]** DeepSeek-V3.2 (671B) and gpt-oss 120B fabricated code for every nonexistent npm and crate prompt [dasu2026refusing, p. 6].
- **[V]** By ecosystem, nonexistent npm packages were fabricated on 94% of prompts, Rust crates 87%, Go modules and PyPI modules 63% each, and Java APIs 33% [dasu2026refusing, p. 8].
- **[V]** A misspelled real package name led to fabrication on only 14% of prompts, against 94% for a correctly spelled fake one [dasu2026refusing, p. 8].
- **[V]** 66% of hallucinations were confident fabrications without any caveat [dasu2026refusing, p. 6].
- **[V]** Wording matters more than the model: prompt identity explains 34% of outcome variation against 15% for model identity [dasu2026refusing, p. 8].
- **[V]** One model suggested pip install pywfm, a name that already exists on PyPI as an unrelated 2015 package [dasu2026refusing, p. 9].

Limitations and weaknesses

- **[A]** Template prompts may differ from what developers type, so absolute rates belong to this benchmark [dasu2026refusing, p. 10].

- **[A]** One answer per prompt; the judge sees the note on the fake item, and the human gold set was labelled by one author [dasu2026refusing, p. 10].
- **[A]** Model size is confounded with 4-bit quantization, and closed models were not tested [dasu2026refusing, p. 10].
- **[I]** The user supplies the fake name, so rates such as 94% are not comparable with spontaneous invention rates from other studies.
- **[I]** Package results rest on 22 prompts per model, so per-model package rates are uncertain.
- **[I]** The judge agreed with humans on only 67% of behaviour labels, so behaviour shares are indicative [dasu2026refusing, p. 5].
- **[I]** Publishing the fake names lets anyone register them, which could change the ground truth for future replications.

Gaps and future work this paper leaves open

- **[A]** G1: closed frontier models and agentic settings are not evaluated [dasu2026refusing, p. 10].
- **[A]** G2: the authors recommend checking package names against registries at generation time but do not test it [dasu2026refusing, p. 9].
- **[A]** G4: size is confounded with quantization [dasu2026refusing, p. 10]. The gap report lists this as a hidden confound across studies.
- **[I]** Typo amplification was dropped as a gap in our gap report because this paper and twist2026library already measure it [dasu2026refusing, p. 8].

Why it matters for your research

This is the clearest evidence that larger models do not fix fake-package acceptance. Its benchmark is small and public, and its small models already ran at 4-bit, the setting a laptop uses. A laptop study could re-run the npm, crate and PyPI subsets on local 7B models and test whether a simple registry check before installing reduces harm. Two design habits are worth copying: matched solvable controls and an AI judge validated against human labels.

Connections to other papers

- **[V]** twist2026library finds the same acceptance of fake names, up to 99.22% of tasks [twist2026library, p. 6], as does djire2026evaluating [djire2026evaluating, p. 9].
- **[V]** twist2026library reports one-character typos accepted in up to 25.86% of tasks [twist2026library, p. 6]. Here a typo that the model corrects counts as correct, which may explain the lower 14% [dasu2026refusing, p. 4].
- **[I]** On model size, krishna2025importing finds a size effect [krishna2025importing, p. 6] and zheng2026when none [zheng2026when, p. 5]. The quantization confound here shows why size results disagree (evidence matrix contradiction 3).
- **[V]** The pywfm case is a real-world example of what spracklen2024we warns: a plain existence check passes names that are already registered [spracklen2024we, p. 12].

Library Hallucinations in LLM-Generated Code: A Risk Analysis Grounded in Developer Queries

> **Takeaway:** Ordinary coding requests almost never make AI models invent library names, but asking for a library "from 2025" or mistyping a library name makes many models invent or accept fake ones [twist2026library, p. 5; twist2026library, p. 6]. A tool that lets the model check PyPI helps only if the model actually calls it [twist2026library, p. 8].

Item	Detail
Authors	Lukas Twist, Mark Harman, Helen Yannakoudakis, Jie M. Zhang
Year · Venue	2026 · Proceedings of EMNLP 2026 (main conference)
Status	Peer-reviewed; our copy is the published ACL Anthology version
Identifier	DOI 10.18653/2026.emnlp-main.78 (arXiv 2509.22202)
Paper type	Controlled prompt-variation experiments plus a released benchmark (LibHalluBench)
Relevance to us	High — RQ1, RQ3, RQ4 (main); RQ2 and RQ5 (indirect)

In plain words

Developers often describe the library they want in words ("a fast library", "a library from 2025"), or they misspell its name. This paper asks whether that everyday wording makes AI models invent libraries. It looks at two kinds of error: library name hallucinations (importing a library that does not exist) and library member hallucinations (calling a function that a real library does not have) [twist2026library, p. 1]. The authors present it as the first systematic study of how user-level prompt wording affects these errors [twist2026library, p. 2].

How they did it

1. They started from BigCodeBench, a public coding benchmark, and kept 356 tasks that use 30 libraries without naming them [twist2026library, p. 3].
2. Experiment 1 inserted descriptions mined from 3,917 library questions on the Software Recommendations StackExchange site, such as "fast" or "from 2025" [twist2026library, p. 4].
3. Experiment 2 inserted mistakes: one-character typos, multi-character typos or fake library names, each checked to be unregistered [twist2026library, p. 4].
4. Seven models answered each prompt three times through their providers' APIs, in a fresh session with no system prompt [twist2026library, p. 3; twist2026library, p. 4].
5. Python's own parser read the import lines, and names were compared with PyPI after standard name normalisation [twist2026library, p. 4].
6. Two measures: RHR, the share of responses with a hallucination, and THR, the share of tasks where at least one of the three responses hallucinates [twist2026library, p. 5].

7. Extra studies covered prompt techniques, a PyPI-existence tool and JavaScript and Rust [twist2026library, p. 8; twist2026library, p. 20].

Data and setup

- Models with exact versions: GPT-4o-mini, GPT-5-mini, Ministral-8B, Qwen-2.5-Coder 32B, Llama-3.3 70B, DeepSeek-V3.1 and Claude-4.5-Haiku, each at its default temperature (0.3 to 1.0) [twist2026library, p. 15].
- Model details are recorded as of January 2026 [twist2026library, p. 15].
- The JavaScript and Rust study used 500 tasks and one response per task [twist2026library, p. 20].
- LibHalluBench has 4,173 prompts and is released on Hugging Face (gated) and as a PyPI package [twist2026library, p. 8].
- The detector was checked by hand on 200 responses [twist2026library, p. 15].
- The date of the PyPI name list is not reported.

Key findings

- **[V]** With normal or adjective prompts, library-name hallucination stays at or below 0.31% of responses for every model [twist2026library, p. 5].
- **[V]** "From 2025" requests cause invented library names in 84.74% of tasks for GPT-4o-mini, 63.86% for GPT-5-mini and 61.68% for Qwen-2.5-Coder, but only 0.62% for Claude-4.5-Haiku [twist2026library, p. 5].
- **[V]** Models go along with user mistakes: one-character typos in up to 25.86% of tasks, multi-character typos up to 79.44% and fake names up to 99.22% [twist2026library, p. 6].
- **[V]** Asking for a "hidden gem" gives the highest rate among the adjective prompts, 5.35% [twist2026library, p. 8].
- **[V]** Self-analysis and explicit-check prompts reduced hallucinations in 29 and 30 of 36 settings, while chain-of-thought was inconsistent [twist2026library, p. 19].
- **[V]** With a PyPI tool, GPT-4o-mini called it on 98.65% of "from 2025" responses and hallucinated in 2.49%. GPT-5-mini called it on only 7.89% and gave no code 90.76% of the time [twist2026library, p. 21].
- **[V]** Under user mistakes, invalid libraries were still accepted in up to 38.79% of cases even with the tool [twist2026library, p. 21].
- **[V]** Fake names were also accepted in JavaScript (up to 88.8%) and Rust (up to 96.76%) [twist2026library, p. 20].
- **[V]** 23 of 24 automatically flagged responses were true hallucinations (95.8% precision) [twist2026library, p. 15].

Limitations and weaknesses

- **[A]** A finite set of tasks, ecosystems and model versions [twist2026library, p. 9]; the tool study is a "small case study" [twist2026library, p. 8].
- **[I]** Experiment 2 measures acceptance of the user's mistake, not invention. The threat is typosquatting amplification, not classic slopsquatting.

- **[I]** Any name registered on PyPI counts as valid, so collisions with squatted names are invisible by design [twist2026library, p. 4].
- **[I]** Normalisation cannot map import names to package names (dateutil to python-dateutil); such cases were fixed only by hand [twist2026library, p. 15].
- **[I]** Whether invented names repeat across runs, which matters for slopsquatting, is not reported.
- **[I]** No statistical tests or confidence intervals, mixed temperatures, API models only, and undated registry lists.

Gaps and future work this paper leaves open

- **[A]** G1: the authors name agentic coding workflows as future work [twist2026library, p. 9]. **[V]** The optional tool was rarely called by GPT-5-mini [twist2026library, p. 8].
- **[I]** G2: this supports testing a forced, client-side existence check against a tool the model may skip.
- **[V]** G3: it counts registered import names as valid [twist2026library, p. 4], while raj2026names flags them as squats [raj2026names, p. 8].
- **[I]** G4: no local or quantized models were tested; running LibHalluBench on Ollama models is a cheap extension.
- **[I]** Typo amplification was dropped as a gap because this paper already measures it [twist2026library, p. 6].

Why it matters for your research

This is one of the strongest papers in the set: peer-reviewed, seven models, repeated answers, ablations and everything released. It gives you a ready trigger set that runs on a laptop; year-based and typo prompts are the highest-yield conditions. Its tool study teaches a key design lesson: a check the model may choose to call is weak, so compare it with a check that always runs. Add an explicit import-to-package map and record "registered but suspicious" as its own result.

Connections to other papers

- **[V]** dasu2026refusing finds the same acceptance pattern, with nonexistent npm packages fabricated on 94% of prompts [dasu2026refusing, p. 8].
- **[V]** raj2026names treats registered import names such as pil and faiss as squats, the opposite rule (evidence matrix contradiction 5) [raj2026names, p. 8].
- **[V]** churilov2026range found import names of real packages to be the largest false-positive class [churilov2026range, p. 10].
- **[V]** mirandapena2026empirical also uses BigCodeBench but checks against the installed environment, not PyPI [mirandapena2026empirical, p. 4].

Evaluating Inference-Time Defenses against Package Hallucination in LLM-Generated Code

> **Takeaway:** On eight small open models in four languages, no defence applied at generation time wins everywhere. RAG gives the lowest rate most often but makes JavaScript worse, and plain greedy decoding gives the best balance between fewer fake names and useful answers. The paper also shows that counting standard-library modules as "hallucinations" inflates Python rates by 7.6–9.4 percentage points. Its measurement lessons are strong; its hardware and cost reporting is thin.

Item	Detail
Authors	Alberick Euraste Djire, Iyiola E. Olatunji, Melissa Tessa, Earl T. Barr, Jacques Klein, Tegawendé F. Bissyandé
Year · Venue	2026 · Proceedings of the 41st IEEE/ACM International Conference on Automated Software Engineering (ASE 2026)
Status	Peer-reviewed. Our copy is the arXiv v1 version (it carries the ACM DOI), so page numbers follow that copy
Identifier	DOI 10.1145/3832783.3837555 (arXiv 2608.22652)
Paper type	Comparative empirical evaluation (8 models × 4 languages × 8 decoding conditions, 3 runs each)
Relevance to us	High — RQ1, RQ3 (main), RQ4, RQ5

In plain words

Small open models (1–8 billion parameters, a rough measure of size) can run on modest hardware, and they hallucinate package names more often than big ones [djire2026evaluating, p. 2; djire2026evaluating, p. 4]. The authors ask which inference-time defences work, meaning tricks applied while the model generates, with no retraining. They also fix a measurement mistake. If you check names only against the package registry, standard-library modules (such as os or json, which ship with Python and are never on PyPI) look like fake packages. This paper corrects for that.

How they did it

1. Build prompts. They took the 1,000 most popular packages per registry from Libraries.io, and GPT-4o-mini wrote one coding instruction per package: 4,000 instructions in total [djire2026evaluating, p. 3].
2. Fix the ground truth. They downloaded full registry name lists "as of March 4, 2026" and added standard-library lists [djire2026evaluating, p. 3]. A name counts as hallucinated only if it is absent from both [djire2026evaluating, p. 4].
3. Test defences. Greedy decoding (always pick the single most likely next token), RAG (retrieval-augmented generation: real package descriptions pasted into the prompt, k=5),

Self-Refine (the model critiques its answer up to 5 times), and four decoding tricks (CD, DoLa, ALCD, Nudging) that need the model's internal scores and sometimes a second model [djire2026evaluating, p. 4; djire2026evaluating, p. 5].

4. Measure. Package hallucination rate (PHR) pooled over all names (micro) and averaged per prompt (macro). Package Utility (PU) asks whether the suggested packages are still useful, using similarity to the seed package; the authors call it a proxy [djire2026evaluating, p. 4]. Code quality was checked with Semgrep, a static code checker [djire2026evaluating, p. 7].
5. Stress test. 4,000 extra prompts asked the model to install and use a fake package name [djire2026evaluating, p. 9].
6. Three runs per configuration, reported as mean ± standard deviation [djire2026evaluating, p. 5].

Data and setup

- Eight models: gemma-1b/4b, deepseek-coder-1.3b/6.7b, qwen2.5-1.5b/3b, mistral-7b and llama-3.1-8b; languages Python, JavaScript, Ruby and Rust [djire2026evaluating, p. 3].
- Hardware: only "within the memory budget of a single GPU" [djire2026evaluating, p. 4]. GPU type, inference software, quantization and run times are not reported. No latency or memory cost is given for any defence.
- No significance tests or confidence intervals.
- Code, data and prompts are on Zenodo [djire2026evaluating, p. 11].

Key findings

- **[V]** Treating standard-library modules as hallucinations raises pooled Python PHR from 24.7% to 32.3% (+7.6 pp), and by up to +9.4 pp for one defence [djire2026evaluating, p. 5].
- **[V]** Corrected baseline PHR ranges from 16.1% (Mistral-7b) to 47.9% (Qwen-1.5b), with a peak of 63.1% (Qwen-1.5b, Rust) [djire2026evaluating, p. 5; djire2026evaluating, p. 6].
- **[V]** RAG gives the lowest PHR in 19 of 32 settings and lowers the average from 29.7% to 18.1%. But it raises JavaScript PHR for 7 of 8 models, by 12.7 pp on average [djire2026evaluating, p. 6].
- **[V]** Greedy decoding matches Self-Refine at 25.6% average PHR and gives the highest utility, so it has the best overall trade-off [djire2026evaluating, p. 6; djire2026evaluating, p. 8].
- **[V]** DoLa is the worst method (38.0% average PHR; 83.5–91.2% on DeepSeek-1.3b). The decoding tricks also add syntax errors, up to 29% for DeepSeek-6.7b on Rust [djire2026evaluating, p. 6; djire2026evaluating, p. 7].
- **[V]** Prompts that name a fake package push baseline PHR to 46.6–92.5%; Ruby is the most vulnerable language [djire2026evaluating, p. 9].
- **[V]** Under those prompts, RAG or Self-Refine is best in 29 of 32 settings; the lowest rate is 11.4% [djire2026evaluating, p. 10].

Limitations and weaknesses

- **[A]** Prompts are synthetic and were not checked against real developer questions. Only 8 small models, 4 ecosystems and one setting per defence were tested [djire2026evaluating, p. 10].
- **[I]** Internal inconsistencies. The abstract's RAG count does not match the 19 of 32 stated in the body, and the abstract says rates rise "up to 45 pp" while the conclusion says 58.1 pp [djire2026evaluating, p. 1; djire2026evaluating, p. 6; djire2026evaluating, p. 11]. The text says RAG never raises PHR in Python, Ruby or Rust, but the table shows Mistral-7b on Rust rising from 18.6 to 20.8 [djire2026evaluating, p. 7].
- **[I]** In the results table, greedy equals the sampled baseline exactly for two models in all four languages [djire2026evaluating, p. 7]. This looks like a copy error.
- **[I]** Prompts are built from the most popular packages, so RAG may simply retrieve the seed package. That flatters RAG and may understate hallucination on niche tasks.
- **[I]** There is no release-date cutoff: a name registered after the model was trained, including a squatted hallucination, counts as valid.
- **[I]** In the code-generation extension, imports are checked against the registry with no import-to-package map [djire2026evaluating, p. 6]. Names like cv2 (installed as opencv-python) probably count as hallucinations.
- **[I]** Neither a plain registry filter nor PackMonitor-style constrained decoding was tested.

Gaps and future work this paper leaves open

- **[A]** Make standard-library exclusion a baseline requirement; test indirect prompt injection and multi-turn manipulation; combine guided decoding with lightweight registry verification [djire2026evaluating, p. 11].
- **[I]** G2: registry verification is proposed but not evaluated, and no defence has cost figures.
- **[I]** G3: the paper fixes standard-library handling but not import names or release dates, which shows how scoring rules change the numbers.
- **[I]** G4: small models across four ecosystems are covered, but quantization and local runtimes are not reported [djire2026evaluating, p. 4]. Our G4 was narrowed because of this paper.

Why it matters for your research

This is the most laptop-relevant defence study among the five briefs in this batch. Greedy decoding, RAG and Self-Refine need no access to model internals, and the 1–4B models are the kind that can plausibly run on a laptop. But the authors used a GPU and give no timings, so laptop speed is untested. Copy their measurement rules: a dated registry snapshot, a standard-library list, and both micro and macro rates. Add what they lack: an import-to-package map, a release-date cutoff, and timings on your own machine. Treat their defence rankings as moderate evidence from one study.

Connections to other papers

- **[V]** RAG results disagree across studies: a slight rise in liu2026packmonitor [liu2026packmonitor, p. 13], large drops in Python in spracklen2024we [spracklen2024we, p. 13], and little effect in Rust in zheng2026when [zheng2026when, p. 8] (contradiction 6).
- **[V]** Import-name scoring differs: twist2026library counts registered import names as valid [twist2026library, p. 4], while raj2026names flags registered import names such as pil and faiss as squats [raj2026names, p. 8] (contradiction 5).
- **[V]** Models largely go along with user-supplied fake names in other studies too [twist2026library, p. 6; dasu2026refusing, p. 8].
- **[V]** Frontier models in churilov2026range hallucinate far less (3.63–5.02% of mentions) [churilov2026range, p. 6]. Units and models differ, so do not pool the numbers.

PackMonitor: Towards Zero Package Hallucinations via Decoding-Time Monitoring

> **Takeaway:** PackMonitor is a training-free add-on that stops a model from writing any package name after "pip install" that is not on a fixed PyPI list. On five small open models it brought package hallucination to 0% at a small speed cost. The 0% is largely guaranteed by design, an attacker-registered name still passes, and the method needs direct access to a model's internals; it was only tested on a data-centre GPU server.

Item	Detail
Authors	Xiting Liu, Yuetong Liu, Yitong Zhang, Jia Li, Shi-Min Hu
Year · Venue	2026 · Proceedings of the ACM on Software Engineering (PACMSE, ISSTA 2026 issue)
Status	Peer-reviewed. Our copy is the arXiv v2 version (it carries the ACM DOI), so page numbers follow that copy
Identifier	DOI 10.1145/3832118 (arXiv 2602.20717)
Paper type	Technique proposal with a controlled experiment against baseline defences
Relevance to us	High — RQ3 (main), RQ4, RQ5

In plain words

When an AI coding assistant writes "pip install X", the package X may not exist. This is a package hallucination. An attacker can register that missing name and fill it with malware, which is called slopsquatting. Earlier defences only lowered the hallucination rate. This paper aims to make hallucinated names impossible. The key idea is simple: the set of real PyPI package names is a finite list, so while the model is writing a name, the tool can forbid anything that cannot become a name on that list [liu2026packmonitor, p. 1; liu2026packmonitor, p. 2].

How they did it

1. Watch the output. A parser follows whether the model is writing prose, ordinary code, or the package names after "pip install" inside a code block. It switches the guard on only for those names [liu2026packmonitor, p. 8].
2. Restrict the choices. The PyPI name list is compiled into a DFA (a deterministic finite automaton: a small state machine that accepts exactly the names on the list). A language model writes text one token (a short piece of text) at a time and gives every possible next token a score called a logit. At each step PackMonitor sets the logit of every token that cannot lead to a listed name to minus infinity, so the model cannot pick it. A name may end only once it is a complete listed name [liu2026packmonitor, p. 6; liu2026packmonitor, p. 9].

3. Save time. The DFA is built once offline, stored, and loaded when the model starts [liu2026packmonitor, p. 10].
4. Compare. Baselines were RAG (retrieval-augmented generation: real package names are pasted into the prompt), self-refinement (the model re-checks its own answer), both together, and the fine-tuned models released by an earlier study [liu2026packmonitor, p. 11; liu2026packmonitor, p. 14].
5. Check side effects. Speed per response, and coding skill on HumanEval, a standard set of Python programming problems [liu2026packmonitor, p. 14; liu2026packmonitor, p. 16].

What it needs: the next-token logits and the tokenizer of a model you run yourself. It needs no training and no weight changes [liu2026packmonitor, p. 3; liu2026packmonitor, p. 9]. **[I]** So it cannot be added to closed chat services such as ChatGPT, Claude or Copilot, which do not let you edit token scores at every step.

Data and setup

- Five open-weight models of 7–8B parameters (a rough measure of model size): DeepSeek-Coder-6.7B, CodeLlama-7B, Qwen2.5-Coder-7B, Llama-3-8B and Llama-3.1-8B [liu2026packmonitor, p. 11].
- Prompts came from HFuzzer, a "fuzzer" in which a tester model (DeepSeek-V3) keeps writing new coding tasks for the target model. Each model and setting got one round of 1,000 iterations [liu2026packmonitor, p. 12]. A subset of an earlier prompt set (Package4U) was used for the fine-tuning comparison; its size is not reported [liu2026packmonitor, p. 14].
- Ground truth: 706,618 PyPI names "at the time of our experiments". The snapshot date and name normalisation (for example upper case, "-" vs "_") are not reported [liu2026packmonitor, p. 12].
- Hardware: a server with 8× NVIDIA A100 80 GB GPUs and 1 TB RAM [liu2026packmonitor, p. 12]. There are no CPU or laptop timings.
- Single runs, no statistical tests. Code is public on GitHub [liu2026packmonitor, p. 20].

Key findings

- **[V]** Package hallucination rate (PHR, the share of recommended packages that do not exist) and response hallucination rate fell to 0% on all five models. For example, DeepSeek-Coder went from 8.39% / 11.60% to 0% / 0% [liu2026packmonitor, p. 13].
- **[V]** The other training-free defences left 1.30–7.55% PHR. The lowest value, 1.30% (CodeLlama with RAG plus self-refinement), rests on only 231 recommended packages, against 1,004 without a defence [liu2026packmonitor, p. 13].
- **[V]** RAG can make things worse: Qwen2.5-Coder PHR rose from 3.25% to 3.33% [liu2026packmonitor, p. 13].
- **[V]** The fine-tuned models still hallucinated 2.66% and 10.27% of packages, against 0% for PackMonitor [liu2026packmonitor, p. 14].
- **[V]** With the cached DFA, time per response rose by ×1.07 to ×1.28 (about +0.06 s to +0.58 s), against up to ×2.65 for RAG plus self-refinement. Without the cache it rose by ×2.81 to ×6.24 [liu2026packmonitor, p. 14; liu2026packmonitor, p. 17]. The text calls the worst case "around 0.2s" [liu2026packmonitor, p. 15]; use the table values.

- **[V]** HumanEval scores were unchanged, for example 72.6% vs 72.6% [liu2026packmonitor, p. 16].

Limitations and weaknesses

- **[A]** The guarantee only holds if the name list is correct and complete, so it must be refreshed regularly [liu2026packmonitor, p. 18].
- **[A]** Only chat answers with "pip install" are covered. Requirements files, Dockerfiles, CI configurations and coding agents are not [liu2026packmonitor, p. 19].
- **[I]** The 0% is largely true by construction: the model is forced onto the list, and the paper does not say whether scoring used the same list.
- **[I]** "Valid" does not mean "safe". Any name on PyPI passes, including names attackers have already registered. Vetted allowlists are mentioned but not tested [liu2026packmonitor, p. 6; liu2026packmonitor, p. 19].
- **[I]** When the model would have written a missing name, the mask forces it onto some listed name. Whether that name is the right one is never measured. The paper's own example, "google-protobuf" for the import path google.protobuf, is this case [liu2026packmonitor, p. 6].
- **[I]** Each setting saw a different, self-generated prompt stream: CodeLlama produced 1,004 packages without the guard and 1,514 with it [liu2026packmonitor, p. 13]. With one run per setting, there is no measure of chance variation.
- **[I]** HumanEval cannot reveal harm here: it contains almost no "pip install" lines and was run with greedy decoding, so the guard rarely or never fires [liu2026packmonitor, p. 16].

Gaps and future work this paper leaves open

- **[A]** Extend to conda and npm, private allowlists, requirements files, Dockerfiles, CI and agents; refresh the list over time [liu2026packmonitor, p. 19].
- **[I]** G1: coding-agent workflows are explicitly out of scope [liu2026packmonitor, p. 19], so what agents actually install is untested.
- **[I]** G2: a plain client-side registry or allowlist check gives the same "no missing name gets installed" property without logit access. It was not compared, and neither tool handles names that attackers have already registered.
- **[I]** G3: no snapshot date or normalisation rule is reported, and import-path vs install-name confusion is not handled.

Why it matters for your research

This paper shows that blocking non-existent names is essentially solved when you control the model's decoding. That leaves the harder and more practical problems open. On a laptop with closed API models you cannot use it as published. **[I]** In principle the same idea could be tried with a small local model if your runtime lets you constrain tokens, but the paper never tested this or reported CPU timings. Its main use for you is as a reference point: a cheap client-side check that queries the registry before installing should be compared against this "zero" result. Your

study should also measure what PackMonitor ignores, namely registered lookalikes and whether the substituted package is the right one.

Connections to other papers

- **[V]** It reuses the HFuzzer prompt generator and earlier metrics. HFuzzer itself reported 5.68% PHR for DeepSeek-Coder-6.7B [liu2026packmonitor, p. 12; zhao2025hfuzzer, p. 6].
- **[V]** Its fine-tuning numbers (2.66% / 10.27%) match spracklen2024we, where fine-tuning also lowered HumanEval pass@1 from 51.4% to 25.3% for one model [spracklen2024we, p. 13]. Count these numbers once, from spracklen2024we.
- **[V]** RAG effects disagree across papers: a slight increase here [liu2026packmonitor, p. 13], a rise for JavaScript in djire2026evaluating [djire2026evaluating, p. 6], and large reductions in Python in spracklen2024we [spracklen2024we, p. 13] (contradiction 6 in our evidence matrix).
- **[A]** The same weakness as registry-checked training: names that attackers pre-register count as valid [spracklen2026llm, p. 12; spracklen2024we, p. 12].
- **[V]** Like model editing (liu2026mitigating) and unlearning (spracklen2026llm), it was run on data-centre GPUs [liu2026mitigating, p. 6; spracklen2026llm, p. 8].

Mitigating Package Hallucinations in Large Language Models via Model Editing

> **Takeaway:** A preprint proposing BOUND, a small add-on (731 KB) trained into the parts of a model most responsible for choosing fake package names. On prompts that already made the model hallucinate often, it cut fake names by about 65–80% (relative) while coding scores stayed within ±2%. Still, 9–20% of names stay fake on new prompts, and the method needs model weights and GPU training.

Item	Detail
Authors	Shuhan Liu, Yukai Zhao, Xing Hu, Kui Liu, Xiaohu Yang, Xin Xia
Year · Venue	2026 · arXiv
Status	Preprint (arXiv v1, July 2026), not peer-reviewed
Identifier	arXiv 2607.02052 (DOI 10.48550/arxiv.2607.02052)
Paper type	Technique proposal with a controlled comparison against fine-tuning and model-editing baselines
Relevance to us	Medium — RQ3 (main), RQ4

In plain words

Model editing means changing a small part of a model's internal numbers to fix one specific behaviour, instead of retraining the whole model. This paper treats package hallucination as a boundary problem: in a given task, the model should be able to tell real package names from invented ones [liu2026mitigating, p. 1; liu2026mitigating, p. 2]. The method finds where in the model that decision is made and trains a small LoRA add-on there. LoRA (low-rank adaptation) adds a few small trainable matrices while the original model stays frozen.

How they did it

1. Find hard prompts. Each prompt was asked 5 times and kept as "high-risk" if more than half the answers contained a fake name [liu2026mitigating, p. 5].
2. Locate the responsible parts. One pass through the model plus a gradient (a measure of how much each internal number affects the outcome) scores every module; the top 5 are chosen [liu2026mitigating, p. 3; liu2026mitigating, p. 4; liu2026mitigating, p. 6].
3. Edit. A LoRA add-on is trained on those modules. It rewards the real names the model itself produced, penalises its own fake names, and keeps other behaviour close to the original [liu2026mitigating, p. 4; liu2026mitigating, p. 5].
4. Validity rule. A name counts as real only if it exists on PyPI and had a release before the model's knowledge cutoff [liu2026mitigating, p. 5].

5. Compare with full fine-tuning, Self-Refinement with registry feedback, and three other editing methods (ROME, MEMIT, DINM), all using the same edit cases [liu2026mitigating, p. 5; liu2026mitigating, p. 6].

Data and setup

- Three open models: deepseek-coder-6.7b-instruct, Qwen3-8B and Llama-3.1-8B-Instruct [liu2026mitigating, p. 6].
- Prompts: the recent Python subset of the prompt set from spracklen2024we. Per model there were 609–1,134 high-risk prompts, split into 4 edit sets of 50 and 409–934 "unseen" prompts [liu2026mitigating, p. 5].
- Five samples per prompt at temperature 0.7 [liu2026mitigating, p. 6].
- Hardware: a server with 8× NVIDIA A100 80 GB GPUs [liu2026mitigating, p. 6].
- Not reported: the PyPI query date, the cutoff date per model, standard-library handling, code or data release, and variance.

Key findings

- **[V]** On edit prompts, the share of fake names fell from 0.4463 to 0.0897 (−79.9% relative) [liu2026mitigating, p. 6].
- **[V]** On unseen high-risk prompts it fell from 0.4421 to 0.1529 (−65.4%), against −37.8% for full fine-tuning [liu2026mitigating, p. 6; liu2026mitigating, p. 7].
- **[V]** The remaining share of fake names on unseen prompts was 0.0927–0.1955, depending on the model [liu2026mitigating, p. 7].
- **[V]** Full fine-tuning beat BOUND in one case: on DeepSeekCoder edit prompts, 0.0188 vs 0.0745 fake-name share [liu2026mitigating, p. 7].
- **[V]** Some baselines "succeed" only by collapsing: ROME on Llama left valid answers in just 1.1% of cases (Valid-Rate 0.0110) [liu2026mitigating, p. 7].
- **[V]** The effect carried over partly to other tasks: −12.8% in code generation and −34.0% in "pip install" recommendation (relative) [liu2026mitigating, p. 8].
- **[V]** HumanEval coding scores changed by under 2%, for example 68.96% to 67.58% [liu2026mitigating, p. 9].
- **[V]** Cost per edit run (unit not defined): 23.81 s to locate, 83.09 s to edit, with a 731 KB add-on, against 14.27 GB for full fine-tuning [liu2026mitigating, p. 9].

Limitations and weaknesses

- **[A]** Regex extraction may miss names; only three open models, PyPI only, and three tasks were tested [liu2026mitigating, p. 9; liu2026mitigating, p. 10].
- **[I]** The test set is extreme: only prompts with base rates around 85% were kept. Relative cuts on this subset do not say how much the rate would fall on ordinary prompts.
- **[I]** "Unseen" prompts come from the same filtered pool. If the same invented names recur, part of the gain may be suppression of memorised names. Name overlap is not reported.

- **[I]** Qwen3 got worse at giving valid answers on unseen prompts (0.7990 to 0.7434) [liu2026mitigating, p. 7], and its baseline HumanEval score of 23.29% is unexpectedly low [liu2026mitigating, p. 9].
- **[I]** In code generation, an unedited model maps imports to PyPI names, so a language model sits inside the measurement [liu2026mitigating, p. 7].
- **[I]** The claim of lower edit time leaves out DINM, which is faster [liu2026mitigating, p. 9].

Gaps and future work this paper leaves open

- **[A]** More ecosystems and languages (e.g. Java); configuration files, build scripts and repository-level workflows [liu2026mitigating, p. 10].
- **[I]** G3: its release-date cutoff is one of only two in the set [liu2026mitigating, p. 5; krishna2025importing, p. 3]. A shared protocol should adopt it, with the dates reported.
- **[I]** G2: 9–20% of names stay fake, so a client-side check before installing is still needed.
- **[I]** G1: agents and repository workflows are not covered.

Why it matters for your research

You cannot run this on a laptop without a GPU. The editing step trains through 7–8B models, and the authors used A100s. **[I]** The small add-on could in principle be loaded by a local runtime for inference, but this was not tested. Closed API models cannot be edited at all. The most reusable idea is the validity rule: count a name as real only if it existed before the model's cutoff. That stops later-registered names, including attacker-registered ones, from counting as correct. Treat the effect sizes as weak evidence: one preprint, no variance, and extreme prompts.

Connections to other papers

- **[I]** Three authors (Zhao, Hu, Xia) also wrote HFuzzer (zhao2025hfuzzer) [liu2026mitigating, p. 1].
- **[V]** PackMonitor reaches 0% without training [liu2026packmonitor, p. 13], while BOUND leaves 9–20% on unseen prompts. Prompts and scoring differ, so compare with care.
- **[V]** spracklen2026llm also changes model weights, but with full-parameter unlearning on data-centre GPUs [spracklen2026llm, p. 8].
- **[V]** Earlier full fine-tuning lowered HumanEval pass@1 from 51.4% to 25.3% for one model [spracklen2024we, p. 13]. BOUND's small add-on avoids that loss in this paper's tests.

LLM Ghostbusters: Surgical Package Hallucination Suppression via Adaptive Unlearning

> **Takeaway:** A preprint that retrains a whole model to "unlearn" its own fake package names while reinforcing real ones. On two DeepSeek-Coder models the rate fell from 21.23% to 2.56% and from 27.75% to 6.13% on 40 held-out paraphrased prompts. It needs 12–24-hour full fine-tuning runs on data-centre GPUs. For you, its most useful part is the warning that attacker-registered names poison any check based only on the registry.

Item	Detail
Authors	Joseph Spracklen, Pedram Aghazadeh, Farinaz Koushanfar, Murtuza Jadliwala
Year · Venue	2026 · arXiv (cs.CR)
Status	Preprint (arXiv v2, September 2026), not peer-reviewed
Identifier	arXiv 2605.01047 (DOI 10.48550/arxiv.2605.01047)
Paper type	Technique proposal with a controlled comparison against other unlearning methods, plus ablations
Relevance to us	Medium — RQ3 (main), RQ4, RQ2

In plain words

Machine unlearning means training a model to stop producing specific outputs. Here the outputs to forget are invented package names. The method, Adaptive Unlearning (AU), works in a loop: the model writes code, a registry check finds the fake names, and the model is trained to make those names less likely while keeping the real names and the surrounding code unchanged. No human labelling is needed [spracklen2026llm, p. 1; spracklen2026llm, p. 2]. Once trained, the fix lives in the model's weights, so answering costs no extra time [spracklen2026llm, p. 12].

How they did it

1. Sample answers to hallucination-prone prompts, check every package name against a PyPI snapshot, and mark each token as context, real name or fake name [spracklen2026llm, p. 5; spracklen2026llm, p. 7].
2. Train with two pulls at once: reinforce real names with an ordinary training loss, and push fake names down with NPO (negative preference optimisation, a loss that lowers chosen outputs relative to a frozen copy of the original model) [spracklen2026llm, p. 5].
3. Keep the signal fresh. When a prompt stops producing fake names, or after 5 rounds, the model rewrites the original prompt to create a new variant [spracklen2026llm, p. 7].
4. Detect names in two ways: parse "pip install" lines, and ask the model which packages are required (Mode 1) and which would be useful (Mode 2) [spracklen2026llm, p. 8; spracklen2026llm, p. 21].

5. Compare with gradient ascent (GA, simply pushing the fake outputs down), NPO alone and PMC, all through the same pipeline [spracklen2026llm, p. 8].

Data and setup

- Two models: DeepSeek-Coder-7b-instruct-v1.5 and DeepSeek-Coder-V2-Lite-Instruct (16B in total, about 2.4B active) [spracklen2026llm, p. 8].
- 20 training prompts, chosen because they make the model hallucinate. The test set is 40 prompts: two ChatGPT paraphrases of each training prompt [spracklen2026llm, p. 8].
- Ground truth: a static PyPI snapshot taken before the experiments [spracklen2026llm, p. 21]. Snapshot date, standard-library handling and the number of names behind each rate are not reported.
- Coding skill was measured on EvalPlus (HumanEval and MBPP coding problems with extra tests) [spracklen2026llm, p. 9].
- Hardware: AMD MI210/MI300 data-centre GPUs [spracklen2026llm, p. 8]. Training is full-parameter (every weight changes), not LoRA, with runs capped at 24 h (7B) and 12 h (16B) [spracklen2026llm, p. 19]. The number of GPUs is not reported.
- Single runs. Code is in an anonymous repository [spracklen2026llm, p. 13].

Key findings

- **[V]** DeepSeek-7B: the hallucination rate fell from 21.23% to 2.56%, against 7.2% for GA, 14.17% for NPO and 11.84% for PMC [spracklen2026llm, p. 10].
- **[V]** DeepSeek-16B: the rate fell from 27.75% to 6.13% [spracklen2026llm, p. 10].
- **[V]** Coding skill rose slightly on 7B (EvalPlus average 66.98 to 68.57) but fell on 16B (77.4 to 73.17; HumanEval 81.1 to 73.2). GA kept the 16B average at 77.35 [spracklen2026llm, p. 10].
- **[V]** On 7B, GA already reached 7.2% with unchanged coding skill and much less drift from the original model (code KL divergence 0.0031 vs 0.0796) [spracklen2026llm, p. 10]. KL divergence measures how far the changed model's outputs move from the original.
- **[V]** Both pulls are needed: reinforcement alone left 12.06% and suppression alone 12.42% [spracklen2026llm, p. 11].
- **[V]** Rewriting prompts helps: 10.7% without it, falling with more rewrites [spracklen2026llm, p. 12].
- **[A]** If attackers register hallucinated names, the registry check calls them valid, and AU "reinforces what it was deployed to suppress" [spracklen2026llm, p. 12].

Limitations and weaknesses

- **[A]** The check only tests whether a name exists, not whether it is used correctly, and packages hosted outside PyPI may be mislabelled [spracklen2026llm, p. 12].
- **[A]** Only two DeepSeek models and Python were tested; frontier-scale fine-tuning exceeded the compute budget [spracklen2026llm, p. 12].

- **[I]** The test prompts are paraphrases of the training prompts, so this is close to an in-distribution test. With the number of names unreported, 2.56% may rest on few names.
- **[I]** The comparison is not evenly tuned. AU reports "the checkpoint that yielded best results" with no stated validation set, while GA and NPO report their final checkpoint at a lower learning rate [spracklen2026llm, p. 19].
- **[I]** Numbers disagree inside the paper: 12.7% vs 10.7% for no rewriting, and a restart at checkpoint 60 vs "50 restarted" [spracklen2026llm, p. 7; spracklen2026llm, p. 12; spracklen2026llm, p. 18; spracklen2026llm, p. 19].
- **[I]** "Surgical" is overstated. The result is a trade-off: lower rates bought with more drift (7B) or lower coding scores (16B).
- **[I]** There is no standard-library filter or release-date cutoff, so modules like os may count as fake, and later-registered names count as real.

Gaps and future work this paper leaves open

- **[A]** Other ecosystems (npm, vcpkg, Conan), other model families, and checks of semantic correctness [spracklen2026llm, p. 12; spracklen2026llm, p. 13].
- **[I]** G2: oracle poisoning means a guard must look beyond "does the name exist" and use age, history or a vetted list.
- **[I]** G3: a static, undated snapshot without standard-library handling is exactly the scoring variation our protocol gap describes.
- **[I]** G5: how serious the poisoning risk is depends on how often hallucinated names actually get registered over time, which is still thinly measured.

Why it matters for your research

This is not laptop work. It needs full fine-tuning of 7B/16B models on MI210/MI300-class GPUs for half a day or more, and it cannot be applied to closed API models; the authors pitch it to model vendors [spracklen2026llm, p. 13]. Use it for two things. First, as background: training-based fixes can lower but not remove fake names. Second, as an argument for your design: a check that only asks "is this name registered?" is weak, because attackers can register names. Treat its numbers as weak evidence (one preprint, single runs, paraphrased test set).

Connections to other papers

- **[I]** It shares authors with spracklen2024we and builds on its prompt benchmark [spracklen2026llm, p. 8]. In that earlier paper, full fine-tuning lowered HumanEval from 51.4% to 25.3% for one model [spracklen2024we, p. 13]; here the 7B model kept its coding skill.
- **[V]** liu2026mitigating also changes weights, but with a small LoRA add-on and a release-date validity rule that this paper lacks [liu2026mitigating, p. 5].
- **[I]** PackMonitor has the same poisoning weakness: any name on the list passes [liu2026packmonitor, p. 6]. spracklen2024we made the same point about plain registry checks [spracklen2024we, p. 12].

- **[V]** So far, real registration is rare: 25 of 2,282 hallucinated names within about six months in churilov2026range [churilov2026range, p. 12], and publicly listed names were claimed mostly by defensive placeholders in okwor2026hallucination [okwor2026hallucination, p. 5].

Names Can Hurt: Spotting Slopsquatting Risks Caused by Package Name Hallucinations in Local Coding LLMs

> **Takeaway:** A small preprint that builds a cheap guard: look each suggested package up on PyPI, score suspicious registered ones with a metadata classifier, and retry when needed. With one small Llama model, 76% of 300 prompts finished without a flagged name. Its headline, that half the flagged names are already registered lookalikes, is probably inflated by normal import names such as pil and faiss. The guard needs no GPU, and despite the title the models ran on a hosted API, not locally.

Item	Detail
Authors	Akash Raj, Sargam Sahu
Year · Venue	2026 · arXiv
Status	Preprint (arXiv v1, August 2026), not peer-reviewed. The metadata DOI is a Zenodo DOI; our metadata check returned a mismatch, and the PDF's identity is only "probable" because it prints no arXiv id
Identifier	arXiv 2608.23897 (metadata DOI 10.5281/zenodo.22087562)
Paper type	System design with a small empirical evaluation and a convenience-sample user survey
Relevance to us	High — RQ3 (main), RQ2, RQ1, RQ4, RQ5

In plain words

The authors want a client-side guard: a tool on the developer's own machine that stops hallucinated Python package names from turning into slopsquatting installs (an attacker registering a hallucinated name and filling it with malware). They target small open-weight models of 7B to 70B parameters [raj2026names, p. 2]. The guard first asks PyPI whether the name exists. If it does, a second layer asks whether the registered package looks like a low-quality lookalike. The authors are an M.Tech student and an intern, and the work is small, but it is the only tested guard in our set that needs no access to model internals.

How they did it

1. Extract names. Python code blocks are parsed with an AST (abstract syntax tree, the program's structure as Python itself reads it), and "pip install" lines are read only inside code fences [raj2026names, p. 3; raj2026names, p. 5].
2. Layer 1. One PyPI lookup per name: "not found" (HTTP 404) means fake [raj2026names, p. 5].
3. Layer 2. A Random Forest (a standard classifier that combines many decision trees) scores registered names with 10 features: 8 about the shape of the name, plus maintainer count and whether a GitHub link exists [raj2026names, p. 5].

4. Import names. A 24-entry table maps import names to install names (cv2 to opencv-python, PIL to Pillow). It is used only after a 404 [raj2026names, p. 5; raj2026names, p. 6].
5. Retry. A flagged answer triggers a retry at higher temperature, then a hand-off to a larger fallback model; every attempt is logged [raj2026names, p. 6].
6. A survey of 24 students rated the tool's interface [raj2026names, p. 11].

Data and setup

- 300 hand-written one-line Python prompts in 18 tiers, from routine tasks to adversarial "bait" prompts [raj2026names, p. 6].
- Models: llama-3.1-8b (primary) and llama-3.3-70b (fallback), both hosted on Groq's free tier [raj2026names, p. 2]. Hardware, quantization and model versions are not reported, and no model ran on the authors' own machine.
- Classifier data: 464 unique names from 7 models, leaving only 2–12 fake examples per test slice [raj2026names, p. 9; raj2026names, p. 10]. How these were labelled is not reported.
- Standard-library handling is not reported: every import is looked up, and every 404 counts as fake [raj2026names, p. 5].
- Code is said to be on GitHub [raj2026names, p. 15]; we did not check it.

Key findings

- **[V]** 186 of 300 prompts gave clean code at once. 86 (28.7%) used up their retries, and the fallback rescued 14 of 85. In total 228 of 300 (76%) finished clean [raj2026names, p. 7].
- **[V]** Most improvement came from fixing extraction bugs: the give-up rate went from 79% to 57% to about 29% across versions, measured on different prompt sets [raj2026names, p. 7]. **[A]** Changing the classifier "did not help much" [raj2026names, p. 12].
- **[V]** Give-ups rise with prompt type: 0–10% on routine tiers, 55% and 73.3% on bait tiers (5–30 prompts each) [raj2026names, p. 7; raj2026names, p. 8].
- **[V]** Of 179 flagged names, about half (90 flags) were registered on PyPI. 36 of these were pil and 10 were faiss [raj2026names, p. 8; raj2026names, p. 9]. **[I]** Both are normal import names of real projects (Pillow, faiss-cpu), so many "registered lookalikes" are probably correct imports, not invented names.
- **[V]** The classifier reduces to two metadata features (maintainers 57.6%, GitHub link 24.5% of importance). A two-feature rule matched it, F1 0.99 vs 0.98 [raj2026names, p. 10; raj2026names, p. 12]. F1 is a 0–1 score that balances missed and false alarms.
- **[V]** The first data split leaked: 72.9% of test rows contained names already seen in training. After the fix, F1 was 0.833–1.000 with very wide uncertainty [raj2026names, p. 9; raj2026names, p. 10].
- **[V]** Falling back to a bigger model of the same family failed again in 83.5% of cases, with the same invented names (pdfgenius, ultrarag) recurring [raj2026names, p. 10].

Limitations and weaknesses

- **[A]** The PyPI JSON API shows maintainers only to admins, so the top feature is basically 0 in live use, risking false alarms on small legitimate packages [raj2026names, p. 13].
- **[A]** Brand-new legitimate packages will be mis-flagged, and none of the drift fixes is implemented [raj2026names, p. 14]. The ≥30 registered names were not audited, and survey participants were the intern's friends [raj2026names, p. 13; raj2026names, p. 14].
- **[I]** There is no hand-checked ground truth: "hallucinated" means "flagged by the detector", so false alarms count as hallucinations.
- **[I]** The claim of cutting give-ups "from 79% (unmitigated)" compares against a buggy early version on a different prompt set [raj2026names, p. 7; raj2026names, p. 15].
- **[I]** Inconsistencies: bait refusal is 6 of 10 in the abstract but 9 of 10 in the text, and the two-feature rule is written as both AND and OR [raj2026names, p. 1; raj2026names, p. 11; raj2026names, p. 12].
- **[I]** One model pair, one provider, Python only, single runs, no confidence intervals except for F1.

Gaps and future work this paper leaves open

- **[A]** Cross-family fallback, richer features such as first-upload age, more training examples, drift handling, and squats with realistic metadata [raj2026names, p. 13; raj2026names, p. 14].
- **[I]** G2: this is the only black-box guard in our set, and its weak points (import names, new legitimate packages, realistic-looking squats) are exactly the hard cases an independent benchmark must contain.
- **[I]** G3: extraction and import-name handling dominated its results, which shows why a shared measurement protocol is needed.
- **[I]** G4: genuinely local inference, for example quantized models, was not tested.
- **[I]** G1: the retry loop resembles what an agent does after a failed install; repeating invented names is the behaviour to watch for.

Why it matters for your research

This is the paper closest to a laptop-only project. The guard is one PyPI lookup plus a rule the authors say fits in "one line of Python" [raj2026names, p. 12]. It needs no GPU and works with any model, including closed APIs. Use it as your baseline defence, but fix its weaknesses: filter standard-library modules, keep a separate import-to-package map, record PyPI status separately from "hallucinated", replace the maintainer feature with upload age or release history, and hand-label a sample to measure precision. Treat its numbers as weak evidence.

Connections to other papers

- **[V]** Import-name scoring is contested: twist2026library counts registered import names as valid [twist2026library, p. 4], and churilov2026range found import names to be the largest false-positive class [churilov2026range, p. 10] (contradiction 5).

- **[A]** spracklen2024we argued that plain existence checks are inadequate because attackers can register names [spracklen2024we, p. 12]; this paper's second layer is an attempt to answer that.
- **[V]** Recurring names fit earlier findings: 43% of hallucinated names recur in all 10 re-queries [spracklen2024we, p. 9]. This paper lists pkg-resources as an invented name [raj2026names, p. 8], while zhao2025hfuzzer found pkg_resources (a module that ships with setuptools) recommended by all nine models [zhao2025hfuzzer, p. 7].
- **[I]** It credits Spracklen et al. with the term "slopsquatting" [raj2026names, p. 2], while twist2026library attributes it to Park (2025) [twist2026library, p. 1].
- **[V]** The white-box defences need data-centre GPUs [liu2026packmonitor, p. 12], and only one study ran its pipeline on a laptop-class machine [mirandapena2026empirical, p. 17].

An Empirical Analysis of Static Analysis Methods for Detection and Mitigation of Code Library Hallucinations

> **Takeaway:** Free code-checking tools such as Pyright catch some, but far from all, cases where AI-written Python uses library features that do not exist, and the authors put the realistic ceiling at about 77% [mirandapena2026empirical, p. 8; mirandapena2026empirical, p. 12]. The tools check against installed packages, not PyPI, so they cannot spot a squatted package.

Item	Detail
Authors	Clarissa Miranda-Pena, Andrew Reeson, Cécile Paris, Josiah Poon, Jonathan K. Kummerfeld
Year · Venue	2026 · arXiv
Status	Preprint; the PDF states acceptance in Transactions on Machine Learning Research (TMLR), which we have not verified
Identifier	arXiv 2604.07755 (DOI 10.48550/arxiv.2604.07755)
Paper type	Comparative evaluation of detection, repair and constrained decoding, with manual annotation
Relevance to us	Medium — RQ3, RQ4, RQ5 (main); RQ1 (partial)

In plain words

Static analysis means checking code without running it, a bit like a spell-checker for programs. Mypy and Pyright are free static checkers for Python. Here a library hallucination is code that would only work if imagined functions or arguments of a library existed [mirandapena2026empirical, p. 1]. The paper asks three things: how many of these errors such tools catch, whether an AI can repair them using the tools' messages, and whether a grammar can stop the model from writing them in the first place [mirandapena2026empirical, p. 2].

How they did it

1. Six models wrote code for three Python benchmarks: DS-1000, ODEX and BigCodeBench [mirandapena2026empirical, p. 5].
2. Four detectors were compared: Mypy, Pyright, a grammar built automatically from library documentation, and o3-mini as an AI judge [mirandapena2026empirical, p. 4].
3. The authors hand-annotated 600 failure cases to see what the tools could catch in principle [mirandapena2026empirical, p. 6]. Annotators agreed well, with Cohen's kappa (agreement corrected for chance) of 0.79 [mirandapena2026empirical, p. 7].
4. Repair: o3-mini fixed the code given the error alone or the error plus Mypy or Pyright output [mirandapena2026empirical, p. 9; mirandapena2026empirical, p. 10].

5. Constrained decoding: the grammar limits which tokens the model may write, run in llama.cpp, a tool for running models locally [mirandapena2026empirical, p. 4; mirandapena2026empirical, p. 11].

Data and setup

- Benchmarks: DS-1000 (1,000 problems), ODEX (550 of 945 used) and BigCodeBench (1,140) [mirandapena2026empirical, p. 5].
- Models: Claude-3, GPT-4 and GPT-3.5 through APIs, and Gemma2 2B, Granite 3B and Qwen3 8B as open models [mirandapena2026empirical, p. 5]. Exact API versions are not reported.
- The whole study ran on an M2 laptop-class computer with 24 GB of memory [mirandapena2026empirical, p. 17].
- Detection takes about 1 s with Mypy, 2.5 s with Pyright and 4 s with o3-mini per sample [mirandapena2026empirical, p. 19].
- A code release is not reported.

Key findings

- **[V]** Pyright's recall on library hallucinations (the share it flags) ranges from 0.14 to 0.85 by model and benchmark [mirandapena2026empirical, p. 20].
- **[V]** In the hand-checked sample, the static tools flagged hallucinations for the right reason in only 37.6% (DS-1000), 4.0% (ODEX) and 10.0% (BigCodeBench) of cases [mirandapena2026empirical, p. 8].
- **[V]** The share the tools could catch in principle is 77.0%, 70.5% and 48.5% [mirandapena2026empirical, p. 8].
- **[V]** Benchmark noise is large: ambiguous inputs or outputs explain 36.9% of sampled hallucinations in ODEX and 34.1% in BigCodeBench, but 1.1% in DS-1000 [mirandapena2026empirical, p. 9].
- **[V]** Repair makes more code run, for example Gemma2 2B on DS-1000 from 47.2% to 77.9% [mirandapena2026empirical, p. 9].
- **[V]** Its effect on hallucinations depends on the benchmark. Their share of remaining errors falls on DS-1000 for every model (claude-3 9.1 to 2.7) but rises on ODEX for small models (Qwen3 8B 31.3 up to 55.6) [mirandapena2026empirical, p. 10].
- **[A]** The authors summarise repair as "no improvement" in most cases [mirandapena2026empirical, p. 10]. **[I]** Their own table shows clear gains on DS-1000, so this overstates.
- **[V]** Constrained decoding gives small, mixed changes and adds about 12 seconds per response [mirandapena2026empirical, p. 11].

Limitations and weaknesses

- **[A]** Python only; the grammar cannot handle unusual import aliases, and incomplete documentation causes grammar errors [mirandapena2026empirical, p. 16; mirandapena2026empirical, p. 8].

- **[A]** After the agreement check, one annotator labelled the rest [mirandapena2026empirical, p. 7].
- **[I]** A missing module is judged against the benchmark environment, not a registry [mirandapena2026empirical, p. 18]. Uninstalled real packages and invented ones are merged.
- **[I]** Package-level cases are not reported separately; library causes are only 6.0–14.7% of the sample [mirandapena2026empirical, p. 8].
- **[I]** Two abstract ranges (8.1–40% and 16–70%) could not be matched to any table [mirandapena2026empirical, p. 1].
- **[I]** The repair measure is relative, so it can change when other errors change. No confidence intervals and mixed temperatures.

Gaps and future work this paper leaves open

- **[V]** G2: this is the only study in our set run on a laptop-class machine [mirandapena2026empirical, p. 17]. **[I]** It shows a client-side check is feasible, but a registry layer is still needed to catch squatted names.
- **[I]** G3: its environment-based rule for missing modules is another of the inconsistent definitions the gap report describes [mirandapena2026empirical, p. 18].
- **[A]** The authors suggest tracking data and control flow in static tools and building benchmarks that accept several valid solutions [mirandapena2026empirical, p. 7; mirandapena2026empirical, p. 9].

Why it matters for your research

It proves that a defence study can run on an ordinary laptop. Mypy or Pyright can be a cheap first screen before running AI code. Its limits are clear, though: a squatted package installs fine and passes these checks, so a PyPI or npm lookup must sit on top. Two reporting lessons: count hallucinations in absolute numbers, not only as a share, and hand-check a sample for benchmark noise. Constrained decoding does not look worth its cost here.

Connections to other papers

- **[V]** White-box defences that edit the model or its outputs need model weights and GPU servers [liu2026packmonitor, p. 12; liu2026mitigating, p. 6; spracklen2026llm, p. 8]; this paper's tools do not.
- **[V]** zhang2025llm also defines a missing package by the environment rather than PyPI [zhang2025llm, p. 6].
- **[V]** twist2026library uses the same BigCodeBench tasks but checks names against PyPI [twist2026library, p. 4].
- **[I]** Its checks run before code executes, in line with ohm2020backstabbers' finding that most malicious packages act at install time [ohm2020backstabbers, p. 11].

Backstabber's Knife Collection: A Review of Open Source Software Supply Chain Attacks

> **Takeaway:** Before AI coding tools existed, this study of 174 real malicious packages showed that most attacks used lookalike names and most ran their harmful code the moment the package was installed [ohm2020backstabbers, p. 11; ohm2020backstabbers, p. 12]. It explains why simply installing a hallucinated package name can be dangerous.

Item	Detail
Authors	Marc Ohm, Henrik Plate, Arnold Sykosch, Michael Meier
Year · Venue	2020 · Lecture Notes in Computer Science (proceedings chapter)
Status	Peer-reviewed (LNCS proceedings chapter); our copy is the arXiv version; a 2020 background paper that predates AI coding assistants
Identifier	DOI 10.1007/978-3-030-52683-2_2 (arXiv 2005.09535)
Paper type	Descriptive review: hand-curated dataset of real malicious packages plus attack trees
Relevance to us	Medium — RQ2 (main), RQ3 and RQ5 (background)

In plain words

A software supply chain attack hides malicious code in a package that other people's software depends on. Packages come from registries such as npm (JavaScript), PyPI (Python) and RubyGems (Ruby). Typosquatting means registering a name close to a popular package so that people install it by mistake. The authors collected real malicious packages from these registries, analysed them by hand and drew attack trees, diagrams of the different ways an attack can be carried out [ohm2020backstabbers, p. 1; ohm2020backstabbers, p. 2]. Slopsquatting is a newer variant of the same attack in which an AI model, not a human typo, supplies the wrong name.

How they did it

1. They searched the Snyk vulnerability database, security advisories and research blogs in mid-2019 and updated the search in January 2020 [ohm2020backstabbers, p. 4].
2. Registries had removed the malicious code, so they recovered it from old mirrors, internet archives and research repositories [ohm2020backstabbers, p. 4].
3. They analysed each package by hand: when it runs, under what conditions, how it got in, what it tries to do and whether it hides its code [ohm2020backstabbers, p. 4; ohm2020backstabbers, p. 5].
4. They built two attack trees, one for getting malicious code into a dependency tree and one for running it [ohm2020backstabbers, p. 7; ohm2020backstabbers, p. 9].
5. Results are descriptive shares and averages, with no statistical tests [ohm2020backstabbers, p. 11].

Data and setup

- 469 malicious packages were identified, 59 proof-of-concept packages were excluded, and code was obtained for 174 [ohm2020backstabbers, p. 9].
- Code was obtained for 109 of 374 npm packages, 28 of 44 PyPI packages, 37 of 41 RubyGems packages and none of 10 Maven packages [ohm2020backstabbers, p. 9; ohm2020backstabbers, p. 10].
- The dataset is on GitHub but shared only on justified request, for ethical reasons [ohm2020backstabbers, p. 10].
- The number of people coding the packages and their agreement are not reported.

Key findings

- **[V]** 56% of packages start their malicious code at install time; on PyPI the share is 96% [ohm2020backstabbers, p. 11].
- **[V]** 61% used typosquatting. The mean edit distance (number of single-letter changes) to the imitated name is 2.3 [ohm2020backstabbers, p. 12].
- **[V]** Some reuse the exact name of a package from another repository, such as python-sqlite from the Linux apt repository [ohm2020backstabbers, p. 12].
- **[V]** Trojan horses, new packages with no lookalike target, are 11% overall [ohm2020backstabbers, p. 13].
- **[V]** Malicious packages stayed available for 209 days on average before public reporting (median 67) [ohm2020backstabbers, p. 11].
- **[V]** 41% run only under certain conditions, which helps them evade analysis sandboxes (isolated test environments) [ohm2020backstabbers, p. 12].
- **[V]** Data theft is the most common goal, at 55% [ohm2020backstabbers, p. 18]. Targets include SSH keys, npm credentials and environment variables [ohm2020backstabbers, p. 13].
- **[A]** A new malicious package "has to be discovered and referenced" by users before it can do harm [ohm2020backstabbers, p. 7].
- **[A]** The authors recommend version pinning and disabling install scripts, enforced by default [ohm2020backstabbers, p. 16].

Limitations and weaknesses

- **[A]** The dataset leans towards npm, and no packages could be obtained for Java or PHP [ohm2020backstabbers, p. 18].
- **[A]** About 34% are droppers whose second-stage download may be gone [ohm2020backstabbers, p. 18].
- **[A]** The attack trees are not complete [ohm2020backstabbers, p. 4].
- **[I]** Only 28 PyPI packages, so PyPI shares rest on very few cases.
- **[I]** Only detected and retrievable attacks are included; undetected ones are missing.
- **[I]** Hand coding without any agreement check, and per-registry shares are read from bar charts.
- **[I]** The data end in 2019, before AI coding tools, and registry defences have changed since.

Gaps and future work this paper leaves open

- **[V]** G1: install-time execution means the install itself is the harmful event [ohm2020backstabbers, p. 11]. **[I]** Measuring generated text alone therefore misses the moment of harm.
- **[A]** G2: its user-side countermeasures (pinning, disabled install scripts) support a lightweight guard on the user's machine [ohm2020backstabbers, p. 16].
- **[V]** G5: packages stayed live for 209 days on average [ohm2020backstabbers, p. 11]. **[I]** Watching hallucinated names for six months or less can miss later abuse.
- **[I]** Its attack tree has no branch for discovery through AI recommendations, which is exactly the route slopsquatting uses.

Why it matters for your research

This paper gives you the threat model. Slopsquatting is a "create new package" attack in which the AI performs the discovery step the authors saw as the attacker's main obstacle [ohm2020backstabbers, p. 7]. Because most malicious packages run on install, a defence must act before installation. For a safe study, never install unknown packages; use read-only registry data. The typosquat edit-distance profile can help set thresholds for a lookalike check.

Connections to other papers

- **[V]** okwor2026hallucination confirmed offline that install-time code runs through setup.py and npm postinstall scripts [okwor2026hallucination, p. 7].
- **[V]** churilov2026range found 25 of 2,282 hallucinated names registered within about six months, one with malware [churilov2026range, p. 12].
- **[V]** zheng2026when found 86.48% of invented Rust crate names within 5 edits of a real crate [zheng2026when, p. 7], close to the typosquat distances here.
- **[I]** Its python-sqlite case resembles the import-name confusion that twist2026library and raj2026names handle in opposite ways [twist2026library, p. 4; raj2026names, p. 8].

Research Gaps and Opportunities

A research gap is an open question. The evidence from the analysed papers must show that it is open, and a novelty check must fail to find a study that already answers it. A novelty check is a set of targeted searches that asks: "has someone already done this?" All our checks ran on 2026-11-12. They used Crossref and arXiv (two free scholarly databases) plus web searches for very recent and non-academic work. OpenAlex, a third database, had used up its free daily limit, and Semantic Scholar and Google Scholar were not searched. So "we found no study" means only that these searches found none. It does not mean that no such study exists.

Papers found during the novelty checks are not part of our 17 analysed papers. We saw only their abstracts. Statements about them are author claims **[A]**. We name them in plain text with their identifiers, so you can look them up. Vendor reports and blog posts seen during the searches guided the searching but are never used as evidence.

How the gaps were ranked **[I]**: by how new the gap is, how feasible it is on a laptop, and how much a result would matter, given your situation (beginner, laptop only, no GPU, short project). These rankings are our judgement.

Gap	In short	Novelty verdict	Confidence	Beginner feasibility	Estimated cost and time
G1	How often AI coding agents actually try to install invented or suspicious packages during ordinary tasks, and what they do when the install fails	novel for the narrowed claim	medium	moderate	up to about $100 in API fees plus 20–50 hours of unattended runs; 4–6 weeks
G2	An independent comparison of laptop-friendly pre-install guards on the hard cases	partially novel	medium	easy to moderate	$0; 2–3 weeks
G3	A validated, shared rule for deciding whether a package name is invented	novel	medium-high	easy	$5–30; 3–4 weeks including labelling
G4	The effect of local model compression levels (GGUF files run with Ollama) on invented PyPI and npm names	partially novel (an increment)	medium-low	moderate	$0; 40–80 hours of unattended runs
G5	Watching published invented names for 12 months or more	mostly addressed (narrow remainder)	low	easy	$0; 1 day setup plus a monthly check

[I] Confidence here means how sure we are that the gap is real and still open. G3 is ranked third by the gap report but should be done first, because G1, G2 and G4 all need its scoring tool (see the recommended project below).

G1. Do AI coding agents actually install invented packages during ordinary tasks?

> **Gap.** We found no study that measures how often AI coding agents actually run pip install or npm install on names that do not exist (or look suspicious) during ordinary, non-adversarial tasks, or what the agents do after such an install fails.

An AI coding agent is an AI tool that does not just suggest code but also runs commands by itself, such as installing packages. Non-adversarial means that nobody is trying to trick the agent: the tasks are normal everyday coding requests.

Why it is a gap.

- **[A]** Five analysed papers from different groups say agents are not covered or are future work [churilov2026range, p. 14; dasu2026refusing, p. 10; twist2026library, p. 9; liu2026packmonitor, p. 19; okwor2026hallucination, p. 8]. This makes it the most often stated gap in the set.
- **[V]** Installation is the harmful step, so measuring only the text the AI writes misses the point. 56% of real malicious packages run their code during installation, and 96% of the PyPI ones do [ohm2020backstabbers, p. 11]. Install-time code execution through pip and npm was confirmed offline [okwor2026hallucination, p. 7].
- **[A]** No analysed paper reports real installs. Measurement stops at the generated text or uses a stand-in, such as checking whether a web address answers [haque2025secure, p. 7; zheng2026when, p. 10].
- **[V]** Models write install commands even for names their own code comments call placeholders [zhao2025hfuzzer, p. 9].
- **[V]** A model offered an optional PyPI lookup tool called it on only 7.89% of responses, and invented libraries remained in up to 38.79% of cases [twist2026library, p. 8; twist2026library, p. 21].
- **[V]** When one model gave up, retrying with a bigger model from the same family failed again on 83.5% of cases, and the same invented names came back in both models [raj2026names, p. 10].
- **[V]** Invented names repeat: 43% came back in 10 of 10 repeated queries [spracklen2024we, p. 9]. **[I]** So agent installs of invented names could be predictable, which is what an attacker needs.

Novelty verdict. Novel for the narrowed claim. Confidence: medium. Attacks where someone plants instructions for the agent, and agents confusing their own local setup, are already studied. What we did not find is a natural base rate of real install attempts on ordinary tasks, checked against the live registry, plus a description of what agents do after a failed install (searches on 2026-11-12). G1 is provisional for one more reason, explained under Limitations in About this guide.

Closest prior work (abstracts only, **[A]**):

- Setup Complete, Now You Are Compromised (Bagmar and Saraf; arXiv 2607.15143, preprint). An attacker edits setup files such as the README or requirements list, and the

study tests real agent tools on 12 scenarios. A simple fixed pre-install check closes most of the gap. This is an attack setting, not a natural base rate.

- StateMirror (Shan et al.; arXiv 2610.32648, preprint). In 4,398 natural agent runs, agents often act on tools or packages that are missing from their local machine. This is about the local machine, not the public registry, so slopsquatting exposure is not measured. Read its full text to check.
- SetupBench (arXiv 2507.09063, preprint) and Beyond pip Install (Milliken, Kang and Yoo; SANER 2025, DOI 10.1109/saner64311.2025.00009). Agents that set up software environments. They measure ability, not security, but are useful sources of tasks.
- Beyond Final Code (arXiv 2503.12374, found by web search only). "Module not found" errors are common in agent logs from a well-known benchmark. These are setup errors, not registry checks.

What you could do. **[I]** Produce the first natural base rate, with confidence intervals, of agents trying to install invented, lookalike or very new packages. Add a simple classification of what agents do after a failed install (fix it correctly, switch to a lookalike name, write the code themselves, give up, or repeat). Compare agents with plain chat on the same tasks and the same model. The safe measurement setup you build is itself reusable.

Suggested study design for a beginner. **[I]**

1. Build the safety setup first (see Safety rules below). Every run happens in a throwaway container whose only network route is a local logging package proxy. The proxy forwards only names that exist and were released before a fixed cutoff date, and returns a normal "not found" for everything else.
2. Phase 0, free, about 1 week: search public agent-run records from benchmark submissions for install commands, and score the names with the G3 scorer. Check that the records are available and licensed for reuse.
3. Pick agents: Aider (an open-source command-line agent that works with API models and local Ollama models), plus mini-SWE-agent or OpenHands. Optionally add Claude Code if you have access.
4. Pick models: one cheap frontier API model from the churilov2026range group, and one local model, for example a 7B coding model in Ollama.
5. Write 200 Python tasks (optionally 100 JavaScript tasks) in neutral words, such as "Write and run a script that ... Install whatever you need." Convert them from the public prompt collection released with the Spracklen and Churilov studies, 39,969 prompts in total [churilov2026range, p. 4]. Balance them by prompt source and topic recency, because both shift the rates [churilov2026range, p. 7; spracklen2024we, p. 8]. Use no fake names and no tricks.
6. Run each task 3 times per agent and model. As a baseline, ask the same model the same task in a single chat turn and score it the same way.
7. Measure: the share of tasks with at least one install request for a name missing from the registry (with Wilson 95% confidence intervals, or the rule-of-three upper bound if there are none); the share per install command; the share of each name type (missing, import name used as install name, registered lookalike, registered after the model's training

cutoff, correct); what the agent does after a "not found"; and how often names repeat across runs and agents.

8. Analyse: compare agent and chat on the same task with a paired test (McNemar), and use a logistic mixed model with the task as a random effect. Preregister your hypotheses.

Feasibility: moderate. Docker, a proxy and the agent tools are well documented, and all scoring is offline Python. The hard parts are safe sandboxing and running enough tasks to see events. Phase 0 gives a no-risk start. The gap report estimates that 600 runs per setup would give about 18 events if the true rate is about 3%, and that zero events would bound the rate below 0.5%, which is still a useful result.

Risks.

- The base rate may be very low, so you may see few events.
- Blocking an install may change what the agent does next. Return a normal registry "not found" so the agent cannot tell.
- Agents and models change between versions. Record versions and dates.
- Tasks that happen to need many installs could make the rate look higher than it is in everyday use.
- Import names used as install names are ambiguous. G3 handles this.

G2. An independent test of pre-install guards on the hard cases

> **Gap.** We found no independent evaluation, on a shared labelled test set, of client-side pre-install guards that run on a laptop, focused on the cases that defeat a plain "does this name exist?" check.

A pre-install guard is a small program that checks a package name before it is installed and warns or blocks if the name looks risky. Client-side means it runs on your own machine, not on the registry.

Why it is a gap.

- **[A]** Plain existence checks are not enough, because attackers can register invented names first [spracklen2024we, p. 12]. If they do, a registry check labels the name as valid [spracklen2026llm, p. 12].
- **[I]** PackMonitor lets through any registered name, and its authors did not evaluate a vetted allow-list [liu2026packmonitor, p. 6; liu2026packmonitor, p. 19].
- **[V]** The hard cases are real. Some invented names were registered within about six months, and at least 23 suggested names had earlier hosted malware [churilov2026range, p. 12]. 19 of 22 registered public names were placeholders [okwor2026hallucination, p. 5], and **[A]** a placeholder can be turned malicious in a later release [okwor2026hallucination, p. 8].
- **[V]** Import names of real packages were the largest class of false alarms, 144–207 mentions per model [churilov2026range, p. 10].
- **[A]** Four papers propose a guard but do not test it: a check against the model's training-cutoff date [krishna2025importing, p. 8], registry grounding while the model writes

[dasu2026refusing, p. 9], guided decoding plus a light registry check [djire2026evaluating, p. 11], and a Rust-specific checker [zheng2026when, p. 9].

- **[V]** The only guard we analysed has known problems. Half of its "registered lookalike" flags are the import names pil and faiss [raj2026names, p. 8]. **[A]** Its top signal is about zero through the public PyPI interface, which risks false alarms on small or new legitimate packages, and it does not handle changes over time [raj2026names, p. 13; raj2026names, p. 14].
- **[V]** A model rarely calls an optional lookup tool [twist2026library, p. 8]. **[I]** This argues for a check that always runs, not one the model may skip.
- **[V]** White-box defences need model weights or internal scores and GPU servers [liu2026packmonitor, p. 12; liu2026mitigating, p. 6; spracklen2026llm, p. 8]. Only static analysis ran on a laptop-class machine [mirandapena2026empirical, p. 17].

Novelty verdict. Partially novel. Confidence: medium. Individual guards exist (a demo tool, a detector preprint, and several community tools). What we did not find is an independent head-to-head comparison of their false alarms and misses on a shared set of hard cases (searches on 2026-11-12).

Closest prior work (abstracts only, **[A]**):

- Bayesian-Calibrated Detection of Hallucinated Package Imports (Hillah et al.; arXiv 2606.13918, preprint). Adds PyPI metadata (age, number of releases, author, summary) to spot registered but suspicious packages, on 1,734 Python snippets from 6 models. It tests how well its scores are calibrated, not a comparison across tools. The closest academic work and a good baseline.
- SlopsGuard (Kurnaz et al.; NOMS 2026 demo, DOI 10.1109/noms69089.2026.11668187). A pre-install tool for several registries. The abstract reports no false-alarm or miss rates. **[V]** It relies partly on lookalike checks, yet 48.6% of invented names are six or more characters away from any real name [spracklen2024we, p. 11].
- Package Hallucinations as Phantoms (Huang et al.; ACM TOSEM 2026, DOI 10.1145/3830237). Proposes NymGuard, which works on the registry side, not as a client-side guard.
- Setup Complete, Now You Are Compromised (arXiv 2607.15143, preprint). A fixed pre-install check helps in attack scenarios but is not compared with existing guards.
- Hal-Detector (Cyber-AI 2026, DOI 10.1109/cyber-ai70573.2026.11755213) and ConfuScope (IEEE TSE 2026, DOI 10.1109/tse.2026.3721700). A JavaScript name classifier and a package-confusion detector. No abstract was retrieved for ConfuScope; check it before finalising.
- Community tools seen on the web report their own false-alarm figures on different sets, so they cannot be compared.

What you could do. **[I]** Build an open, labelled test set of install requests, run the first independent comparison of existing guards on it, and add a minimal reference guard (a pip and npm wrapper plus an agent hook) with tests that show which signals matter. The result is practical advice for developers who use AI agents.

Suggested study design for a beginner. **[I]**

1. Build 1,000–2,000 labelled install requests (PyPI first, npm optional) in six classes.
2. Class C1: about 300 popular legitimate packages. Class C2: about 300 legitimate small or new packages (first released less than 90 days ago). C2 tests false alarms.
3. Class C3: about 100 import names used as install names, such as cv2, sklearn, PIL, yaml, bs4, dateutil and faiss, each labelled with its correct install name.
4. Class C4: about 300 invented, unregistered names from your own G1 or G3 runs and from published lists.
5. Class C5: about 50 invented names that are registered (placeholders and squats), re-checked live. Class C6: about 200 known-malicious names from the OSV malicious-packages data. Use names and metadata only; never download them.
6. Compare: a plain "not found" check; a check with standard name normalisation; the two-signal rule from raj2026names, re-built [raj2026names, p. 12]; a training-cutoff date rule [krishna2025importing, p. 3]; a top-N allow-list; community guards run as black boxes; and a reference guard that combines built-in module filtering, an import-to-install name table, a live registry check, the first-release date, an OSV lookup and simple popularity signals.
7. Measure false-alarm rates (C1–C2) and miss rates (C3–C6) per class with Wilson confidence intervals, plus speed per check on your laptop. Re-test after 4 weeks to see how results drift.
8. Finally, plug the best guard into the G1 setup and count how many would-have-installed events it stops, and how many correct installs it wrongly blocks.

Feasibility: easy to moderate. Pure Python and public web interfaces. No AI model is needed except to generate class C4 names. Keep registry and OSV requests to at most one per second per site, as churilov2026range did [churilov2026range, p. 5].

Risks.

- The registry is used both to label the data and inside the guards (circularity). Check classes C3 and C5 by hand.
- Tools change between versions. Record versions and dates.
- Attackers can fake metadata, which was not tested in raj2026names [raj2026names, p. 13]. You could add a small synthetic class built from metadata records only, never published.
- Class sizes do not reflect real-world frequencies, so report results per class.

G3. A shared, validated rule for deciding whether a name is invented

> **Gap.** We found no study that re-scores the same AI answers under the different published counting rules, checks those rules against human labels, and tests whether the known contradictions survive.

Why it is a gap.

- **[V]** Papers treat import names differently. One counts any registered import name as valid [twist2026library, p. 4; twist2026library, p. 15]. Another flags registered import names as

squats [raj2026names, p. 8]. **[I]** A third probably counts unregistered import names such as cv2 as invented [djire2026evaluating, p. 6].

- **[A]** One study rejects reading import lines, because module names do not map to package names [spracklen2024we, p. 6]. **[I]** Its replication reads import lines anyway [churilov2026range, p. 5].
- **[V]** In one case study, 34 of 46 invented names were wrong install names for a correct import [zhao2025hfuzzer, p. 9].
- **[V]** Counting built-in modules as invented adds 7.6 to 9.4 percentage points in Python [djire2026evaluating, p. 5]. Another paper does not report how it handles them [raj2026names, p. 5].
- **[V]** Saved registry lists miscounted and inflated rates by about a fifth compared with a live re-check [churilov2026range, p. 10]. Few papers apply a release-date rule, for example [krishna2025importing, p. 3; liu2026mitigating, p. 5]. **[A]** The public mirror of npm names is unreliable [okwor2026hallucination, p. 3].
- **[V]** Units differ: per prompt [krishna2025importing, p. 4], per mention [spracklen2024we, p. 7], per string including file paths [haque2025secure, p. 8], or per code reference including built-in modules [zheng2026when, p. 4].
- **[V]** In small studies, extraction errors dominate: most early "hallucinations" were extractor mistakes [raj2026names, p. 7].
- **[I]** Contradictions C1 and C2 (see the overview) may be side effects of these choices. Nobody has tested this.

Novelty verdict. Novel. Confidence: medium-high. The evidence is direct and comes from many papers, and no protocol-validation study was found (searches on 2026-11-12). The main risk is not novelty but attention: a methods paper may get less notice than an attack or defence paper.

Closest prior work:

- churilov2026range's live, two-round protocol (in our set). It is the most complete, but its import-name table covers only about 1,157 import names [churilov2026range, p. 14], and it is not checked against human labels.
- Bayesian-Calibrated Detection (arXiv 2606.13918, preprint; abstract only, **[A]**). Calibrates one detector; it does not compare counting rules.
- AI-Induced Supply-Chain Compromise: A Systematic Review (Al-Zofi; Research Square 2025, DOI 10.21203/rs.3.rs-8007192/v1, preprint; abstract only, **[A]**). Summarises definitions; it does not validate a protocol.

What you could do. **[I]** Write a short, versioned checklist for counting invented names, release an open scoring tool (a small Python library), build a human-labelled gold set, and show how much each rule choice moves the rates and whether contradictions C1 and C2 flip. The scorer is then reused in G1, G2 and G4, so none of the work is wasted.

Suggested study design for a beginner. **[I]**

1. Take 1,000 prompts from the released Spracklen and Churilov prompt collection [churilov2026range, p. 4], balanced by source (Stack Overflow or AI-written) and language (Python or JavaScript).

2. Generate one answer per prompt with two cheap API models and one local Ollama model, plus 3 answers for a 200-prompt subset. That is about 3,600 answers.
3. Build seven scorers: six that copy the published rules (spracklen2024we, churilov2026range, krishna2025importing, twist2026library, raj2026names, djire2026evaluating) and one careful rule of your own.
4. Your rule: pull names from install commands and from import lines separately; remove built-in modules; map import names to install names; normalise names the standard PyPA way [twist2026library, p. 15]; check each name live on the registry with its first-release date; and label it as exists, not found, import name of a real package, or first released after the model's training cutoff.
5. Label a stratified sample of 400 names with a second person, and measure agreement with Cohen's kappa.
6. Report each rule's rate (per mention and per answer, with Wilson confidence intervals), its precision and recall against the gold labels, how model rankings change, and whether C1 and C2 flip. Use the churilov2026range rule as the strongest existing baseline.

Feasibility: easy. Deterministic Python, a small API budget and a clear result. A good first project, or the first work package of the combined project.

Risks.

- Some names are hard to label even for people (old aliases, names valid in another language's registry).
- The released prompts may be in the 2026 models' training data [churilov2026range, p. 14].
- Your copies of the published rules may not match the originals exactly. Test them on any published examples.

G4. Does local model compression change invented names on PyPI and npm?

> **Gap.** We found no study that varies the GGUF compression level of the same models, as people run them locally with Ollama, and measures spontaneous invented PyPI and npm names with careful scoring.

Quantization compresses a model by storing its numbers with fewer bits, so it fits on a laptop. GGUF is the file format most local tools use, and Ollama is a popular tool for running such models.

Why it is a gap.

- **[A]** Local compressed models are explicitly left out of the newest measurement [churilov2026range, p. 14].
- **[V]** The only quantization study is about Go and uses a different compression tool. 4-bit Qwen 0.5B invented 96–99% of Go package strings, against 41–54% at full precision [haque2025secure, p. 6]. Its 8-bit results are not consistent, and it counts file paths as packages [haque2025secure, p. 6; haque2025secure, p. 8].

- **[V]** Quantization is hidden in other studies: all open models in spracklen2024we were compressed [spracklen2024we, p. 6], and **[A]** dasu2026refusing mixes size with 4-bit compression [dasu2026refusing, p. 10]. **[I]** Contradiction C3 (model size) may partly be a compression effect.
- **[V]** Small open models are covered, but without varying compression: djire2026evaluating does not report it [djire2026evaluating, p. 4], and okwor2026hallucination runs Ollama models but reports no rate [okwor2026hallucination, p. 3]. **[A]** raj2026names lists local models as untested [raj2026names, p. 14].

Novelty verdict. Partially novel: an increment over one Go study (from Go to PyPI and npm, and to the compression actually used locally). Confidence: medium-low. Existing data from other studies may already contain compressed runs, and the result may be "small effect".

Closest prior work (abstract only, **[A]**):

- Quantize with Confidence? (Afrin et al.; arXiv 2607.14181, preprint). Six compression methods including GGUF on coding models. It measures correctness, maintainability and security of code, not invented packages.

What you could do. **[I]** Give practical advice to laptop developers ("is a 4-bit model riskier than an 8-bit one for dependencies?") and help explain contradiction C3.

Suggested study design for a beginner. **[I]**

1. Pick qwen2.5-coder 1.5B and 7B plus llama3.2 3B at three levels: Q4_K_M, Q8_0 and F16. Use F16 for the 7B model only if your laptop has 32 GB of memory or more.
2. Use 400 Python and JavaScript prompts from the Churilov collection, 3 answers each at temperature 0.7, scored with the G3 scorer.
3. Measure per-mention and per-answer rates with Wilson intervals, the number of names mentioned (because compression can change it [haque2025secure, p. 5]), refusals, repetition, and overlap of names across levels.
4. Analyse with a logistic mixed model (prompt as random effect; compression level crossed with language).

Feasibility: moderate. The code is simple, but about 7,200 answers at 20–40 seconds each take about 40–80 hours of unattended laptop time.

Risks. Ollama's default model tags and chat templates change over time, so pin exact versions. Laptop hardware can give slightly different outputs from run to run. Small models mention few packages, which gives few data points.

G5. Do published invented names get claimed over a year or more?

> **Gap.** The evidence on whether invented names get registered and turned malicious over long periods is thin. What remains open is an independent re-check, over 12 months or more, of invented-name lists that have already been published.

Why it is a gap.

- **[V]** 25 of 2,282 names were registered within about six months, one with malware. The test of whether publishing names leads to registration was too small to detect anything below about 10.5% [churilov2026range, p. 12]. **[A]** The authors propose ongoing monitoring [churilov2026range, p. 15].
- **[V]** 0 of 1,641 unpublished names were registered after only 4 days [okwor2026hallucination, p. 5]. **[A]** The author proposes to keep watching [okwor2026hallucination, p. 8].
- **[A]** spracklen2024we did not register names and withheld its list [spracklen2024we, p. 14].
- **[V]** Before AI assistants, malicious packages stayed available for 209 days on average [ohm2020backstabbers, p. 11]. **[I]** Windows of six months or less can miss them.

Novelty verdict. Mostly addressed; a narrow remainder is open. Confidence: low. A peer-reviewed paper found during the novelty check covers much of the "is it exploited in the wild?" question.

Closest prior work (abstract only, **[A]**):

- Package Hallucinations as Phantoms in Open Source Software Supply Chains (Huang et al.; ACM TOSEM 2026, DOI 10.1145/3830237, peer-reviewed). The authors registered 107 invented names on npm for 6 months: 35.2% of installs were attributed to developers, mostly within 2 weeks, and web-search help raised how often names repeated from 11.2% to 17.3%. This largely fills the broad version of the gap.
- Security-company write-ups that re-checked the Churilov names were seen on the web, but no independent public monitoring feed was found. These are grey sources and not evidence.

What you could do. **[I]** Add a longer-term data point to Churilov's six-month window, as a cheap add-on to G1 or G2 rather than a project of its own.

Suggested study design for a beginner. **[I]**

1. Every month, look up already-published invented names (for example the public names used by okwor2026hallucination and the names published by churilov2026range, where available) through the PyPI and npm metadata interfaces and OSV.
2. For newly registered names, record the first-release date, number of maintainers, description or placeholder text, and OSV status. Never install or run anything.
3. Report "time to registration" curves combined with Churilov's six-month data [churilov2026range, p. 12].

Feasibility: easy. One day of setup and a monthly scheduled job, at no cost. Low impact on its own.

Risks. Published lists lean towards names claimed by defenders [okwor2026hallucination, p. 6]. Metadata alone cannot tell a harmless placeholder from a malicious one. The likely outcome is a handful of events.

The recommended project: G3, then G1, then G2

> **Recommendation.** One coherent laptop study: "Do AI coding agents actually install invented

> packages, and can a simple pre-install guard stop it?"

[I] The gap report recommends combining three gaps in this order:

1. **G3, weeks 1–3.** Build the shared scoring tool and the human-labelled gold set. This is low-risk and publishable on its own if time runs out.
2. **G1, weeks 3–7.** Run benign everyday tasks through Aider (and optionally Claude Code) with one cheap API model and one local Ollama model. Every install request goes through a logging proxy that never runs unknown packages. This gives the first natural base rate of agent install attempts on missing, lookalike or very new names, plus what agents do after a failed install.
3. **G2, weeks 6–9.** Replay the logged install requests, plus a small labelled hard-case set, through existing guards and a reference guard. Report false alarms and misses per class, and speed on the laptop.

The gap report estimates about $100–150 in API fees, with all heavy runs unattended on the laptop and no GPU. The study uses only benign tasks, never registers names, and reports findings through coordinated disclosure. It produces a measurement result, a reusable safe test setup, and a practical defence evaluation. The main risk is a very low base rate. In that case, a tight upper bound plus the guard evaluation is still a solid result.

Before you start, add and analyse the four high-priority papers listed below, so that the novelty of G1 and G2 is re-confirmed against their full texts.

Safety rules (apply from day 1)

[I] These rules come from the gap report and the research report. They are not optional.

- **Never install unknown packages.** The scorer only reads registry metadata. Any agent run (G1) happens in a throwaway container or virtual machine whose only network route is a local logging package proxy (for example devpi or a small simple-index proxy for PyPI, and Verdaccio for npm).
- **The proxy decides what gets through.** It forwards only names that exist and were first released before a fixed cutoff date (for example 2026-01-01). Everything else gets a normal "not found" and is logged as "would have installed". So no unknown or newly registered package code ever runs.
- **Never register any invented name**, not even as a harmless placeholder.
- **Use coordinated disclosure.** Report risky names privately to the PyPI and npm security teams. Do not publish lists of unregistered invented names. The only test of whether publishing names leads to registration was too small to show that publishing is harmless [churilov2026range, p. 12].
- **Known-malicious names are data only.** For G2, use names and metadata from OSV. Never download the packages.
- **Keep registry requests polite.** At most one request per second per site, with caching [churilov2026range, p. 5].
- **Check with a supervisor** about ethics approval and the disclosure plan before the G1 runs.

Follow-up novelty checks

No separate follow-up topic was novelty-checked for this project, so there are no extra novelty-check reports in the reports folder. All novelty checks are recorded in the gap report. The research report recommends re-running the three shared searches (package hallucination, hallucinated packages, slopsquatting) on the day the project starts, because new preprints appear every month.

Recommended next steps

A checklist, in order:

1. **Make the pending decision described under Limitations in About this guide.** Until then, G1 stays provisional.
2. **Add and analyse the four high-priority papers** in the table below (Huang et al., StateMirror, Setup Complete, and Bayesian-Calibrated Detection). Papers are added with rlab add, which checks their metadata, and each one then needs a full-text analysis. Then re-run the G1 and G2 novelty checks against their full texts.
3. **Choose your direction.** Our suggestion is the combined project G3, then G1, then G2. G3 alone is the safest fallback.
4. **Set up the safety rules** before any agent run: container, logging proxy, cutoff date, and no name registration.
5. **First experiment (G3, week 1): build a scorer and score 200 prompts.** Take 50 prompts each from Stack Overflow and AI-written sources, in Python and JavaScript, from the released Spracklen and Churilov collection [churilov2026range, p. 4]. Check its licence first.
6. **Generate answers** with one free local model in Ollama (for example a 7B coding model), and optionally one cheap API model. **[I]** A small local model will give more invented names to work with than a frontier model.
7. **Write scorer version 0 in Python.** Pull names from install commands and import lines separately, remove built-in modules, map import names to install names starting from the table Churilov used [churilov2026range, p. 14], normalise names the PyPA way [twist2026library, p. 15], and look each name up live with caching.
8. **Score every answer twice,** once with a naive rule (any name not found counts as invented) and once with your scorer. Report rates with Wilson 95% confidence intervals and hand-check about 50 names where the two rules disagree. This pilot tests your setup; it does not answer the research question.
9. **Record tool and model versions and dates** for every run, because AI tools change quickly.
10. **Re-run the novelty searches** on the day the project starts, and again before you write a paper.
11. **Talk to a supervisor** about the statistics (mixed-effects models), ethics approval and coordinated disclosure.
12. **Optional:** an open-access published version of zhang2025llm exists. Replacing our arXiv copy needs your permission.

Recommended papers not yet included (13)

Found during the gap novelty checks on 2026-11-12. Not analysed and not used as evidence. Priority is our judgement **[I]**.

Priority	Paper	Identifier	Why it matters
high	Huang et al., Package Hallucinations as Phantoms in Open Source Software Supply Chains: An Empirical Security Analysis (ACM TOSEM 2026, peer-reviewed)	DOI 10.1145/3830237	Real-world npm registration and install experiment; NymGuard; narrows G5 and informs G2
high	Bagmar and Saraf, Setup Complete, Now You Are Compromised: Weaponizing Setup Instructions Against AI Coding Agents (2026, preprint)	arXiv 2607.15143	Closest agent-install work (attack setting); a fixed pre-install check; bounds G1 and G2
high	Shan et al., StateMirror: Pre-Execution Detection of Hallucinated Tool Calls in Coding Agents (2026, preprint)	arXiv 2610.32648	4,398 natural agent runs, including missing packages; must be read to confirm G1
high	Hillah et al., Bayesian-Calibrated Detection of Hallucinated Package Imports in AI-Assisted Code (2026, preprint)	arXiv 2606.13918	Metadata signals for registered but suspicious names; closest G2 baseline
medium	Kurnaz et al., SlopsGuard: A Defense Tool Against AI-Hallucinated Package Attacks in Software Supply Chains (NOMS 2026 demo)	DOI 10.1109/noms69089.2026.11668187	An existing pre-install guard to compare in G2
medium	Li et al., ConfuScope: Robust Package Confusion Detection (IEEE TSE 2026)	DOI 10.1109/tse.2026.3721700	Package-confusion detector; possible G2 baseline (no abstract retrieved)
medium	Afrin et al., Quantize with Confidence? An Empirical Study of Quantization for Code Generation (2026, preprint)	arXiv 2607.14181	GGUF and other compression methods for coding models; bounds G4
medium	Milliken, Kang and Yoo, Beyond pip Install: Evaluating LLM Agents for the Automated Installation of Python Projects (SANER 2025)	DOI 10.1109/saner64311.2025.00009	Agent installation tasks; a task source for G1
medium	Arora et al., SetupBench (2025, preprint)	arXiv 2507.09063	Environment set-up benchmark; a task source for G1

low to medium	Bonfanti et al., Hal-Detector: Is Explainable AI Helping Detect Slopsquatting? (Cyber-AI 2026)	DOI 10.1109/cyber-ai70573.2026.11755213	JavaScript invented-name classifier
low to medium	Beyond Final Code: A Process-Oriented Error Analysis of Software Development Agents (2025, preprint; found by web search only)	arXiv 2503.12374	How common "module not found" errors are in agent logs; G1 phase 0
low	Al-Zofi, AI-Induced Supply-Chain Compromise: A Systematic Review (Research Square 2025, preprint)	DOI 10.21203/rs.3.rs-8007192/v1	Background review
low	Logroño and Salazar, slopsquatting in npm code (in Spanish; MQRInvestigar 2026)	DOI 10.56048/mqr.2026.e396	Small npm measurement (320 generations, 4 models)

Glossary

Term	Plain meaning
Abstract-only	We read only the paper's short summary, not the whole paper. All 17 analysed papers were read in full.
Agent (AI coding agent)	An AI tool that takes actions by itself, such as running commands and installing packages, not just suggesting code.
Allow-list	A list of names that are approved in advance. Anything not on the list is blocked or flagged.
API	A defined way for one program to talk to another. "API model" means a commercial AI model used over the internet, paid per use.
arXiv	A free public website where researchers post papers, often before peer review.
Baseline	The thing you compare against, such as the plain model with no defence.
Benchmark	A fixed set of tasks and scoring rules used to compare AI systems.
Black-box method	A method that only needs the model's text output, so it also works with closed commercial models.
Cohen's kappa	A measure of how well two people agree when labelling, corrected for agreement by chance. 1 means perfect agreement; 0 means chance level.
Confidence interval	A range that probably contains the true rate. Wider means more

(Wilson 95%)	uncertain. Wilson's method works well for small counts.
Container (Docker)	An isolated, throwaway environment for running software, so nothing it does touches the rest of your computer.
Contradiction	Studies reporting results that disagree.
Coordinated disclosure	Reporting a security problem privately to those who can fix it before making it public.
Crate	A Rust package. crates.io is the Rust package registry.
Data-centre GPU (A100, MI210, MI300)	Expensive graphics processors used in server rooms to train and run AI models. Not available on an ordinary laptop.
DFA (deterministic finite automaton)	A simple state machine that accepts exactly a given list of strings, here valid package names.
Edit (Levenshtein) distance	The number of single-character changes needed to turn one name into another.
Evidence matrix	Our table of all findings from the papers, each with its source, label and strength.
Evidence strength	How much we trust a finding (strong, moderate or weak), based on sample size, design and agreement between studies.
False alarm (false positive) / miss (false negative)	A false alarm flags a safe name as risky. A miss lets a risky name through.
Fine-tuning	Training a model further on extra examples to change its behaviour.
Fuzzing	Automatically generating many varied test inputs to find failures.
GGUF / Ollama	A file format for compressed models, and a popular tool for running such models on a laptop.
Gold set	A set of examples labelled carefully by people and used as the reference answer.
Greedy decoding	Always picking the single most likely next word (temperature 0).
Grey literature	Reports, blogs and vendor material outside academic publishing. Not used as evidence in this guide.
HumanEval	A standard set of programming problems used to check that a model can still write correct code.
Import name vs install (distribution) name	In Python, the name you import (for example cv2) can differ from the name you install (opencv-python).
Install-time execution	Code inside a package that runs automatically while the package is being installed, before you ever use it.
LLM (large language model)	The type of AI behind tools like ChatGPT and Claude. It produces text, including code.
Logging proxy	A local server between your machine and the registry that records every install request and can block it.

Logits	The scores a model gives every possible next token (word piece) before it chooses one.
LoRA / model editing	Changing specific behaviours by training a small add-on set of weights instead of the whole model.
Mixed-effects model	A statistical model that handles repeated measurements on the same units, such as several runs of the same task.
Natural experiment	A study that uses a situation the researchers did not create, such as which names happen to get registered.
Novelty check	Targeted searches to see whether a proposed study has already been done.
OSV	A free public database of known vulnerabilities and malicious packages.
Package (dependency)	A reusable piece of third-party code that a project installs and relies on.
Package hallucination	An AI recommending or importing a package name that does not exist in the registry.
Package registry	The public store that install tools download from: PyPI for Python, npm for JavaScript, crates.io for Rust.
Peer review	Checking of a paper by independent experts before it is published.
Persistence	Whether the same invented name comes back when the same question is asked again.
Pilot study	A small trial run that tests the setup before the full study.
Placeholder package	A registered package with no real content, often registered by a defender to stop an attacker from taking the name.
Pre-install guard	A small program that checks a package name before it is installed and warns or blocks if it looks risky.
Preprint	A paper shared publicly before (or without) peer review, often on arXiv. Flagged and weighted lower in this guide.
Preregistration	Publishing your hypotheses and analysis plan before collecting data, so you cannot change them after seeing the results.
PyPA normalisation	The official Python packaging rule for treating differently written package names as the same name.
Quantization	Storing a model's numbers with fewer bits so it runs on smaller hardware, possibly at some cost to quality.
RAG (retrieval-augmented generation)	Looking up relevant documents and giving them to the model along with the question.
Rate per mention / per answer / per prompt	The share of invented items, which depends on what is counted. Rates with different units cannot be compared.

Registry snapshot vs live check	Checking names against a saved copy of the registry's list from one date, or asking the registry at the moment of checking.
Replication	Repeating a study to see whether the result holds.
Research gap	An open question that the evidence shows is unanswered and that a novelty check could not find already answered.
Rule of three	If 0 events are seen in n tries, the true rate is probably below 3 divided by n.
Saturation	The point where new searches keep returning papers you already have.
Self-refinement / self-check prompt	Asking the model to review its own answer, for example "check that every package you used exists".
Slopsquatting	Registering an AI-invented package name and filling it with malware, so that people who follow the AI's advice install it.
Software supply-chain attack	An attack that reaches victims through third-party code their project depends on.
Standard library	Modules built into a language that never need installing.
Static analysis	Checking code automatically without running it, for example with a type checker such as Pyright.
Temperature	A setting that controls how random the model's word choices are.
Typosquatting	Registering a misspelled version of a popular package name to catch typing mistakes.
Unlearning	Training a model to make specific outputs less likely.
White-box method	A method that needs access to the model's internals (weights or logits), so it only works with models you run yourself.

About this guide

How it was made

This guide was put together on 2026-11-12 from the files of the research project, which also started on 2026-11-12. The project used the rlab tool in this workspace for every routine step, so that metadata, citations and page numbers are checked by a program rather than typed from memory.

1. **Scope.** You chose this topic from four candidate security topics. The defaults: papers from 2023–2026 (older supply-chain background allowed), peer-reviewed first, at most 20 papers in the first pass, preprints allowed but flagged.
2. **Search.** 19 discovery searches across five topics (measurement, exploitation, defences, agents and tools, testing and benchmarks) in OpenAlex, Crossref and arXiv. OpenAlex hit its free daily limit partway through.

3. **Screening.** Three search "scouts" checked the results against the inclusion rules. 20 papers were included, the planned maximum. Saturation was judged high.
4. **Full texts.** All 20 full texts were obtained legally: 19 from arXiv or the ACL Anthology, and one openly licensed report from its Zenodo record.
5. **Analysis and critical review.** 17 papers were analysed from their full text with page references. A second reviewer then checked every analysis against the paper, in two batches, and the fixes were applied.
6. **Synthesis.** All findings went into an evidence matrix (172 findings, 7 contradictions, 5 agreements). Findings from preprints were lowered one strength step. The study methods were compared side by side.
7. **Gap analysis.** Five candidate gaps were tested with 27 targeted searches on Crossref and arXiv plus web searches.
8. **Research report.** A plain-language report summarised everything. This guide reorganises the same material for reading: this overview, one brief per paper, and these closing sections.

Every citation in this guide was checked automatically: each one names a paper in the project and a page that exists in our PDF.

Limitations

- **Three papers not analysed.** The project holds three more papers, all about attacks on AI coding agents: wang2026package (package-hallucination attacks on coding agents through instructions hidden in rule files; arXiv 2610.09264, preprint), hsu2026harmless (neutral-looking prompts that steer hallucination through agent "skills"; Findings of EMNLP 2026, our copy from the ACL Anthology) and liu2026trust (steering which dependencies an agent installs through malicious agent skills; arXiv 2605.09594, preprint). Their full texts are in the project, but they were not analysed, because an automatic safety check stopped the analysis output. What to do with them is waiting for your decision: a limited abstract-only analysis, your own reading with notes recorded, or leaving them out with a note. They are not used as evidence anywhere in this guide and have no briefs. They are the closest work to G1, so G1 stays provisional until they are handled.
- **Size.** The review rests on 17 analysed papers. That is enough to map a young field, but it is not a complete systematic review.
- **Preprints.** 10 of the 17 analysed papers are preprints, including one self-published report. Most findings rest on a single study, and almost none has been independently repeated.
- **Search sources.** OpenAlex ran out of its free daily quota, so later discovery searches and all 27 novelty searches used Crossref and arXiv only, plus web search. Semantic Scholar and Google Scholar were not used. Every "we found no study" statement is limited to the sources we used.
- **Search date.** All searches ran on a single day, 2026-11-12. This field produces new preprints every month, so re-check the gaps before you start.

- **Papers found but not analysed.** 13 relevant papers found during the novelty checks (4 of high priority) are not yet added or analysed. What this guide says about them comes from abstracts only. One of them, a peer-reviewed TOSEM 2026 field study, bears directly on RQ2 and has already narrowed G5.
- **Paper versions.** For 6 of the 7 peer-reviewed papers we analysed the arXiv version, not the publisher's final version, so page numbers may differ. An open-access published version of zhang2025llm exists; replacing our copy needs your permission. liu2026packmonitor's arXiv version has a different title from the published one.
- **Metadata.** Our database still lists spracklen2024we as a preprint, although we treat it as peer-reviewed (USENIX Security 2025). The automatic metadata check flags three known mismatches (one title difference and two Zenodo identifiers not listed in Crossref), each explained in the paper's analysis.
- **Who did the reading.** AI agents wrote the analyses. Each was checked by a critical review against the paper text, and cited pages were spot-checked, but not every number was re-checked by a person.
- **Our judgement.** Evidence-strength ratings, gap rankings and priorities are our own ratings **[I]**. Another team might rate some differently.